DIGITAL ASSIGNMENT 3 / REVIEW 3 · SOFTWARE ENGINEERING

Vigil

Uptime and API monitoring that explains its own outages

Name **Vatsal Agrawal**

Registration number **24BAI1355**

Programme **B.Tech CSE (AI & ML), VIT Chennai**

Repository **github.com/vatsal-agra/vigil**

Release **v0.3.0-rc.1 (Review 3 release candidate)**

Date **15 October 2026**

Contents

Part A — Review 3 Evidence

1 Integration, Regression and Mutation Testing 3

2 Version Management and System Building 10

3 Developed Functionalities 15

4 Tools and Technologies Used 17

5 Project Demo 18

Part B — Project Report

6 Problem Statement 19

7 User Stories 20

8 System Architecture and System Design 23

9 Design of Tests 25

Appendix A — Software Requirements Specification 27

Appendix B — Data Flow Diagrams 29

Appendix C — Entity-Relationship Diagram 30

Appendix D — UML Diagrams 31

Appendix E — Code Listing and GitHub Link 34

> Every screenshot in this report was captured from the running system or from the GitHub repository. Terminal screenshots show the real output of the command on their first line. Diagrams are exported from editable draw.io sources committed in `docs/design/diagrams/`.

1 Integration, Regression and Mutation Testing

Vigil is tested at four levels. Each one answers a different question, and each runs in a different place. The backend suites run inside the `api` container against a **real PostgreSQL database and Redis** (an isolated `vigil_test` database and Redis db 15), so nothing is faked below the HTTP layer. Only the monitored targets are simulated, through `httpx.MockTransport`.

Level	Tests	What it proves	Where it runs
Unit	50	Each module in isolation: probe engine, shared queries, hashing and tokens. Exact values, boundaries, error text.	`pytest -m unit`, every PR
Integration	22	Every API route through FastAPI's full stack with the real database, cache and auth, plus a full outage lifecycle across API, worker, database and cache.	`pytest -m integration`, every PR
Regression	7	One test per defect fixed or contract that must never break again; re-run on every change.	`pytest -m regression`, every PR
Mutation	120 mutants	Whether the tests would actually notice a bug: mutmut changes the code and checks a test fails.	On demand / weekly workflow
End-to-end	6	The core user journeys in a real Chromium browser against the running stack.	`pytest e2e`, against `docker compose up`

1.1 Integration testing

The integration suite exercises the system the way a client does: HTTP requests in, JSON out, with real SQL and real cache keys in between. The most important test follows one outage from start to finish. It creates a monitor through the API, runs the worker's probe cycle against a target that fails twice and then recovers, and asserts on what the dashboard API and the public status page report at each step, including that the worker invalidated the API's cached responses.

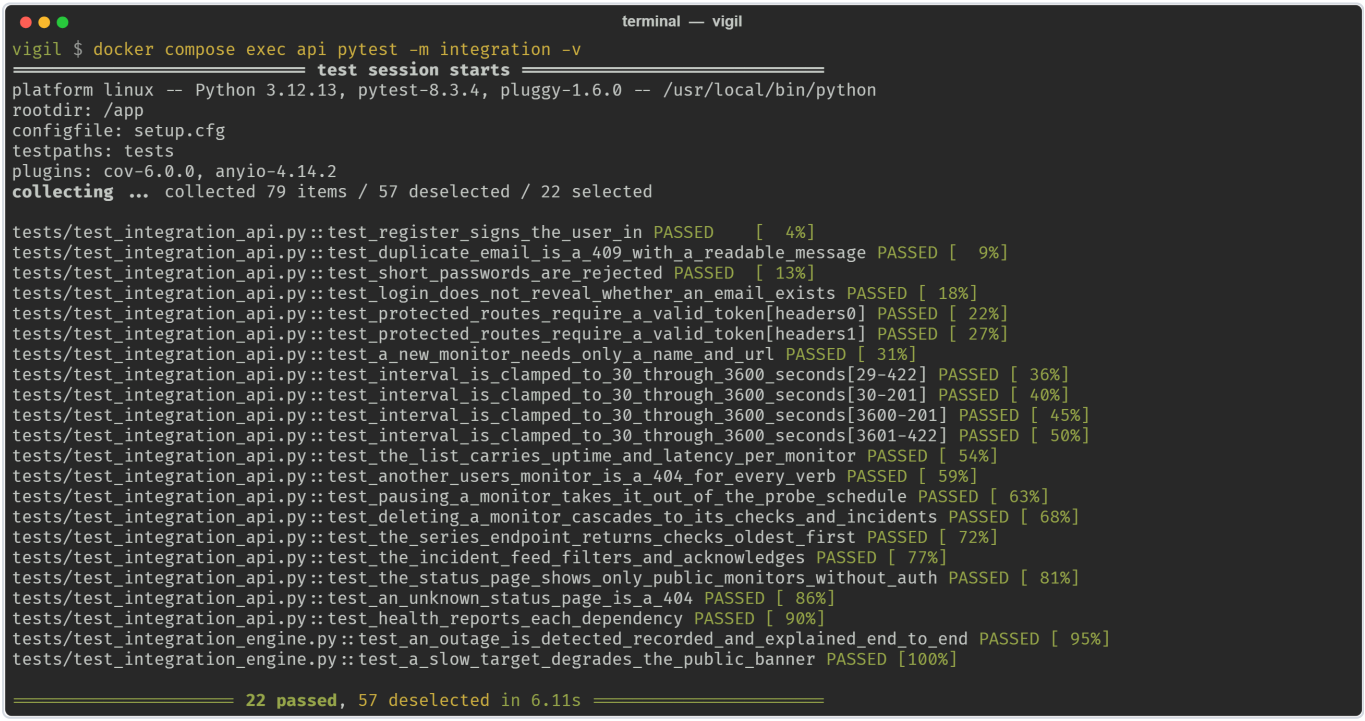


Figure 1.1 — Integration suite: 22 tests through the HTTP API with PostgreSQL and Redis, all passing.

```python
def test_an_outage_is_detected_recorded_and_explained_end_to_end(client, signup, db, fake_web):
    from app.probe import run_cycle
```

```python
    auth, account = signup()
    mid = client.post("/api/monitors", headers=auth,
                      json={"name": "Checkout API", "url": URL, "is_public": True}).json()["id"]

    # warm the caches the way a browser would, so invalidation is exercised too
    assert client.get("/api/monitors", headers=auth).json()[0]["current_status"] == "pending"
    assert client.get(f"/api/status/{account['handle']}").json()["overall"] == "operational"

    # the target starts failing: one failure is tolerated, the second opens an incident
    run_cycle(db, fake_web({URL: 503}))
    assert client.get("/api/incidents?open_only=true", headers=auth).json() == []
    make_due(db)
    run_cycle(db, fake_web({URL: 503}))

    assert client.get(f"/api/monitors/{mid}", headers=auth).json()["current_status"] == "down"
    [incident] = client.get("/api/incidents?open_only=true", headers=auth).json()
    assert incident["cause"] == "Expected 200, got 503"
    assert client.get(f"/api/status/{account['handle']}").json()["overall"] == "major outage"

    # the target recovers: the incident closes itself with a plain-English summary
    make_due(db)
    run_cycle(db, fake_web({URL: 200}))

    assert client.get("/api/monitors", headers=auth).json()[0]["current_status"] == "up"
    assert client.get("/api/incidents?open_only=true", headers=auth).json() == []
    [resolved] = client.get("/api/incidents", headers=auth).json()
    assert resolved["summary"].startswith("Checkout API recovered after")
    assert client.get(f"/api/status/{account['handle']}").json()["overall"] == "operational"
    series = client.get(f"/api/monitors/{mid}/checks", headers=auth).json()
    assert [c["ok"] for c in series] == [False, False, True]
```

backend/tests/test_integration_engine.py, lines 22–54

1.2 Regression testing

Each regression test names the defect or contract it guards, so a red test says exactly what came back. R1 reproduces the Review 2 defect: the dashboard showed "Open incidents: 0" while a monitor was down, because the open incident was older than the newest 100 rows of the capped feed. The suite runs as its own CI step on every pull request.

```
terminal — vigil
vigil $ docker compose exec api pytest -m regression -v
============================= test session starts ==============================
platform linux -- Python 3.12.13, pytest-8.3.4, pluggy-1.6.0 -- /usr/local/bin/python
rootdir: /app
configfile: setup.cfg
testpaths: tests
plugins: cov-6.0.0, anyio-4.14.2
collecting ... collected 79 items / 72 deselected / 7 selected

tests/test_regression.py::test_R1_open_incident_beyond_the_feed_cap_is_still_reported PASSED [ 14%]
tests/test_regression.py::test_R2_a_single_failed_check_never_opens_an_incident PASSED [ 28%]
tests/test_regression.py::test_R3_foreign_resources_are_404_not_403 PASSED [ 42%]
tests/test_regression.py::test_R4_naive_timestamps_do_not_crash_the_scheduler PASSED [ 57%]
tests/test_regression.py::test_R5_the_public_page_never_leaks_urls_or_raw_errors PASSED [ 71%]
tests/test_regression.py::test_R6_interval_bounds_are_enforced_on_update_as_well_as_create PASSED [ 85%]
tests/test_regression.py::test_R7_run_cycle_never_closes_a_client_it_was_given PASSED [100%]

======================= 7 passed, 72 deselected in 2.65s =======================
```

Figure 1.2 — Regression suite (pytest -m regression): R1–R7, all passing.

1.3 Mutation testing

Coverage shows which lines ran, but not whether a test would notice if those lines were wrong. Mutation testing checks that directly. **mutmut** makes 120 small changes ("mutants") to `probe.py` and `services.py`, the two modules holding the business logic. It flips `>=` to `>`, swaps `*` for `/`, replaces `continue` with `break`, and so on, then re-runs the tests for each one. A mutant that every test still passes is a bug the suite would have missed. It runs in a throwaway container built from the image, so the source files on disk are never modified.

Run	Mutants	Killed	Timed out	Suspicious	Survived	Mutation score

Round 1: initial suite	120	92	1	2	25	**79.2%**
Round 2: after mutation-driven tests	120	116	1	1	2	**98.3%**
Final: mutant #48 re-run after one more test fix	120	117	1	1	1 (equivalent)	**99.2%**

The 25 survivors from round 1 were read one by one, and they pointed at real gaps rather than noise:

- **Values never pinned exactly.** Latency was asserted to exist but not to equal anything, so `* 1000` → `/ 1000` survived, as did rounding to 2 decimals instead of 1. A fake `perf_counter` now fixes the clock, and the tests assert `123.5` ms exactly.
- **Boundaries.** `>=` → `>` on "is this monitor due?" and on the 24-hour window survived, because no test sat exactly on the boundary. Frozen-clock tests now do.
- **Untested behaviour.** `continue` → `break` in the scheduler survived because no test had two never-checked monitors. The worker closing its own HTTP client, and following redirects, were never checked.
- **Runner scope.** Two mutants were already caught by regression tests R4 and R7, but the mutmut runner only ran `-m unit`, so it was widened to `unit or regression`.
- **Equivalent mutants.** `.strip("-")` → `.strip("XX-XX")` can never change behaviour, because the input is lower-cased first. Mutants like this cannot be killed and are recorded as such.

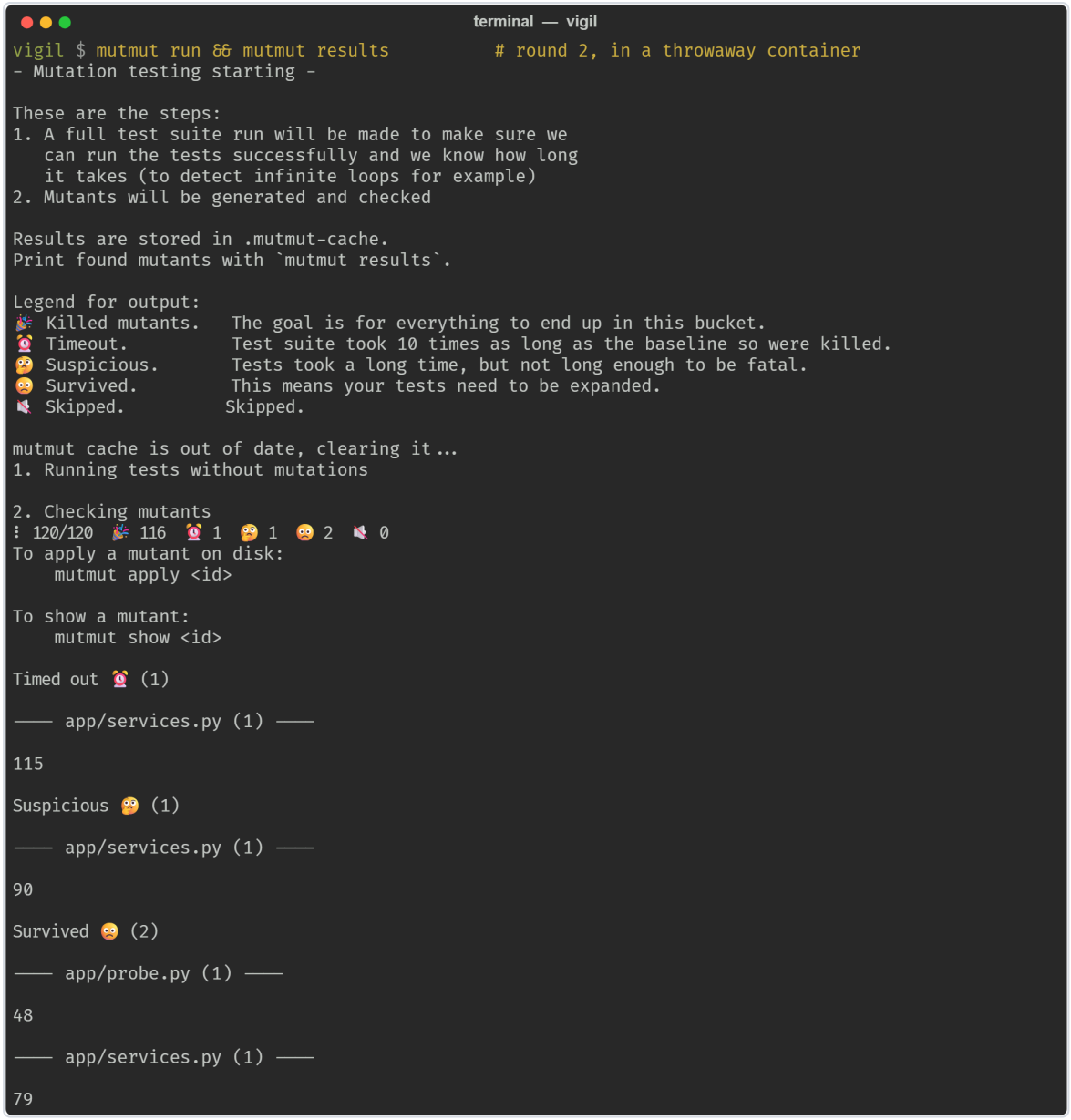


Figure 1.3 — Mutation testing round 2: 120 mutants over probe.py and services.py; 116 killed, 2 survived (spinner frames collapsed to the final line, as a terminal shows them).

Round 2 left two survivors. **#79** is the equivalent `strip` mutant above. **#48** changes the outage length from `/ 60` to `/ 61` minutes. It survived because the new test used a 30-minute outage, and 30 × 60 / 61 still prints as "30 min". The test now uses a 10-hour outage (600 vs 590 minutes). Re-running just that mutant against the saved round-2 state kills it, leaving only the equivalent mutant: **119 of 120 caught (99.2%), with every killable mutant killed.**

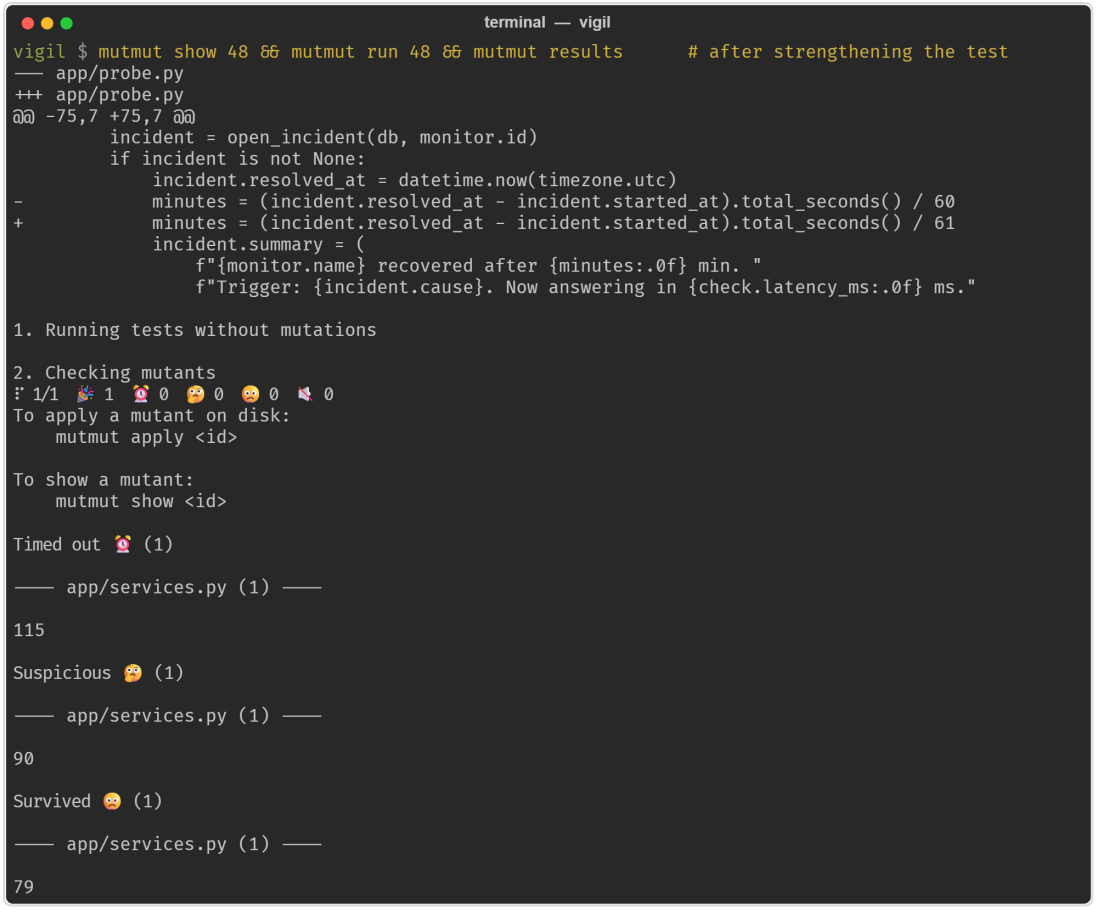


Figure 1.4 — The mutant that survived round 2, re-run after the test fix: killed. mutmut results now lists only the equivalent #79.

1.4 Unit tests, coverage and end-to-end tests

```
terminal — vigil
vigil $ docker compose exec api pytest -m unit -v
tests/test_unit_probe.py::test_one_failure_is_noise_not_an_incident PASSED [ 22%]
tests/test_unit_probe.py::test_the_second_consecutive_failure_opens_an_incident PASSED [ 24%]
tests/test_unit_probe.py::test_continued_failures_do_not_open_duplicate_incidents PASSED [ 26%]
tests/test_unit_probe.py::test_the_failure_threshold_comes_from_settings PASSED [ 28%]
tests/test_unit_probe.py::test_recovery_resolves_the_incident_and_writes_a_summary PASSED [ 30%]
tests/test_unit_probe.py::test_a_success_resets_the_failure_counter PASSED [ 32%]
tests/test_unit_probe.py::test_slow_but_successful_checks_are_degraded_above_two_seconds[150.0-up] PASSED [ 34%]
tests/test_unit_probe.py::test_slow_but_successful_checks_are_degraded_above_two_seconds[2000.0-up] PASSED [ 36%]
tests/test_unit_probe.py::test_slow_but_successful_checks_are_degraded_above_two_seconds[2000.1-degraded] PASSED [ 38%]
tests/test_unit_probe.py::test_every_result_is_stored_and_timestamped PASSED [ 40%]
tests/test_unit_probe.py::test_a_result_invalidates_exactly_the_affected_cache_keys PASSED [ 42%]
tests/test_unit_probe.py::test_run_cycle_probes_only_due_monitors_and_reports_the_count PASSED [ 44%]
tests/test_unit_probe.py::test_run_cycle_with_nothing_due_does_no_work PASSED [ 46%]
tests/test_unit_probe.py::test_latency_is_measured_in_milliseconds_to_one_decimal PASSED [ 48%]
tests/test_unit_probe.py::test_a_timeout_records_its_duration_in_milliseconds PASSED [ 50%]
tests/test_unit_probe.py::test_a_monitor_is_due_at_exactly_its_interval PASSED [ 52%]
tests/test_unit_probe.py::test_every_never_checked_monitor_is_due_not_just_the_first PASSED [ 54%]
tests/test_unit_probe.py::test_the_recovery_summary_reports_the_outage_length_in_minutes PASSED [ 56%]
tests/test_unit_probe.py::test_a_failure_without_an_error_message_still_names_a_cause PASSED [ 58%]
tests/test_unit_probe.py::test_run_cycle_closes_the_client_it_creates_and_follows_redirects PASSED [ 60%]
tests/test_unit_probe.py::test_run_cycle_logs_each_check PASSED          [ 62%]
tests/test_unit_security.py::test_password_round_trip PASSED             [ 64%]
tests/test_unit_security.py::test_hashes_are_salted_pbkdf2 PASSED        [ 66%]
tests/test_unit_security.py::test_a_malformed_stored_hash_is_rejected_not_raised PASSED [ 68%]
tests/test_unit_security.py::test_token_carries_the_user_id_and_an_expiry PASSED [ 70%]
tests/test_unit_services.py::test_rollup_computes_uptime_and_average_latency_of_successes PASSED [ 72%]
tests/test_unit_services.py::test_rollup_ignores_checks_older_than_the_window PASSED [ 74%]
tests/test_unit_services.py::test_rollup_rounds_uptime_to_two_decimals PASSED [ 76%]
tests/test_unit_services.py::test_rollup_of_an_unchecked_monitor_is_empty_not_zero PASSED [ 78%]
tests/test_unit_services.py::test_open_incident_returns_only_the_unresolved_one PASSED [ 80%]
tests/test_unit_services.py::test_open_incident_is_none_when_everything_is_resolved PASSED [ 82%]
tests/test_unit_services.py::test_slugify[Checkout API (v2)!-checkout-api-v2] PASSED [ 84%]
tests/test_unit_services.py::test_slugify[  Spaces  everywhere -spaces-everywhere] PASSED [ 86%]
tests/test_unit_services.py::test_slugify[UPPER-upper] PASSED            [ 88%]
tests/test_unit_services.py::test_slugify[!!!-monitor] PASSED            [ 90%]
tests/test_unit_services.py::test_unique_slug_appends_a_counter_per_owner PASSED [ 92%]
tests/test_unit_services.py::test_unique_handle_derives_from_the_email_and_avoids_collisions PASSED [ 94%]
tests/test_unit_services.py::test_rollup_defaults_to_exactly_24_hours PASSED [ 96%]
tests/test_unit_services.py::test_rollup_includes_a_check_exactly_at_the_window_edge PASSED [ 98%]
tests/test_unit_services.py::test_rollup_rounds_latency_to_one_decimal PASSED [100%]

====================== 50 passed, 29 deselected in 8.03s =======================
```

Figure 1.5 — Unit suite (last 40 lines of output): 50 tests, all passing.

```
terminal — vigil
vigil $ docker compose exec api pytest --cov=app --cov-report=term-missing
.................................................................... [ 91%]
.......                                                              [100%]

---------- coverage: platform linux, python 3.12.13-final-0 ----------
Name                       Stmts   Miss  Cover   Missing
--------------------------------------------------------
app/__init__.py                0      0   100%
app/cache.py                  27      6    78%   21-22, 29-30, 38-39
app/config.py                 13      0   100%
app/db.py                     12      0   100%
app/main.py                   54     12    78%   24-27, 35-39, 72-73, 77-78
app/models.py                 54      0   100%
app/probe.py                  70      0   100%
app/routers/__init__.py        0      0   100%
app/routers/auth.py           26      1    96%   33
app/routers/incidents.py      29      0   100%
app/routers/monitors.py       71      2    97%   32, 114
app/routers/status.py         31      1    97%   21
app/schemas.py                57      0   100%
app/security.py               40      1    98%   60
app/seed.py                   29     18    38%   24-74
app/services.py               28      0   100%
--------------------------------------------------------
TOTAL                        541     41    92%

79 passed in 15.01s
```

Figure 1.6 — Whole backend suite with coverage: 79 passed, 92% line coverage; probe.py and services.py at 100%.

The uncovered lines are the start-up wait loop and the seed script, which run once when the container boots, plus the `except RedisError` branches in `cache.py`. Those branches are exercised by stopping Redis by hand

(documented under the demo), not by the automated suite.

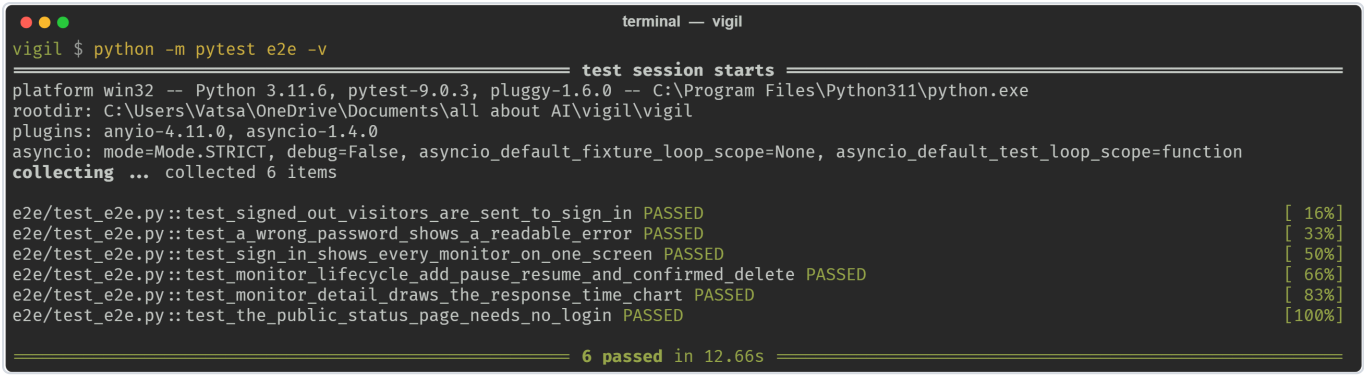


Figure 1.7 — End-to-end tests: Playwright drives Chromium through the six core journeys against the running stack.

Writing the end-to-end tests directly from the acceptance criteria exposed two gaps in US-12. The API supported pausing, but nothing in the interface could pause a monitor (AC1). "Stop watching" deleted immediately without the confirmation AC3 requires. Both were fixed in commit `904790c`, and the E2E test now drives both paths.

2 Version Management and System Building

2.1 Version management

The repository follows **GitHub Flow**. `main` is always deployable, every unit of work happens on a short-lived branch named after its story (`feat/us-25-test-suite`), and changes reach `main` only through a pull request whose CI checks are green. Commits follow **Conventional Commits** (`feat`, `fix`, `test`, `ci`, `docs`, `refactor`) and reference the issue they serve. Each review milestone is a **semantic-version tag** with a GitHub Release: `v0.1.0` (Review 1), `v0.2.0` (Review 2), and `v0.3.0-rc.1`, a pre-release, because Review 3's pull request is still open.

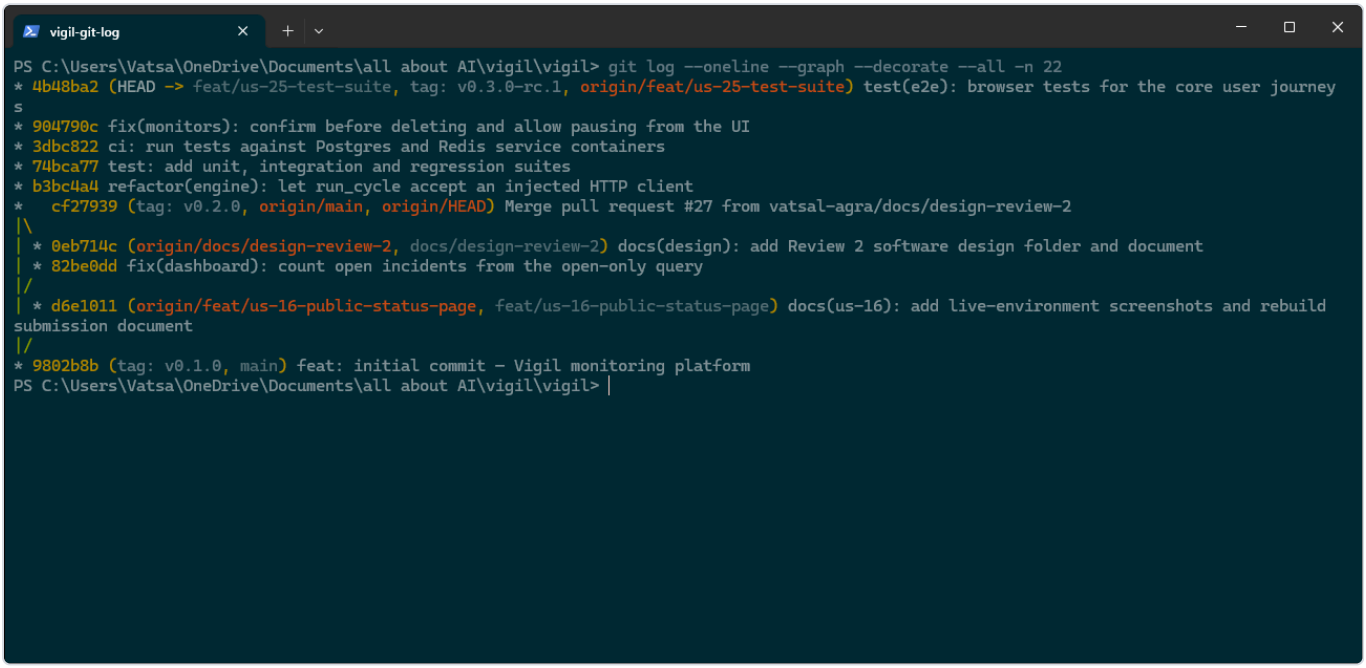


Figure 2.1 — Commit graph: feature branches merged through pull requests, tags at each review.



Figure 2.2 — Local and remote branches, and the annotated release tags.

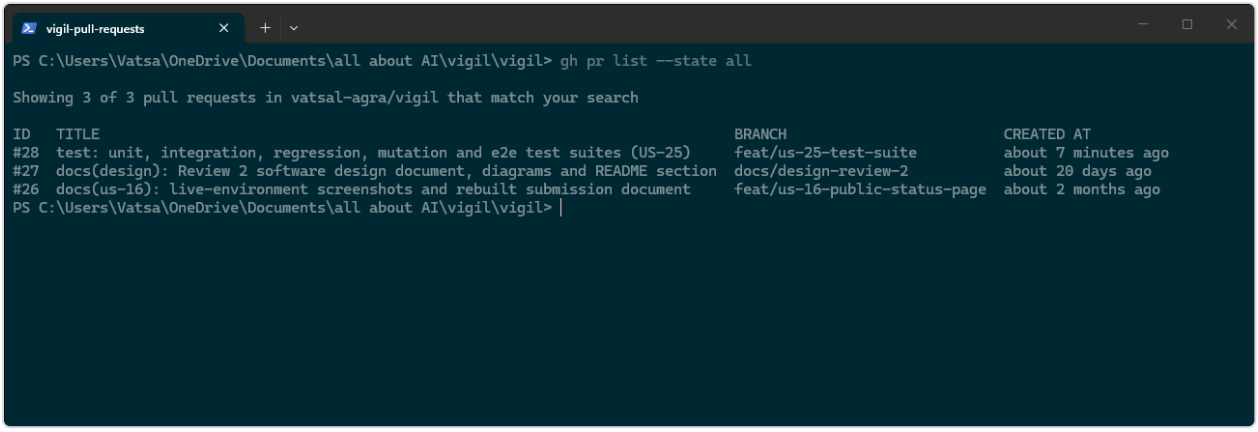


Figure 2.3 — Every change reached main through a pull request.

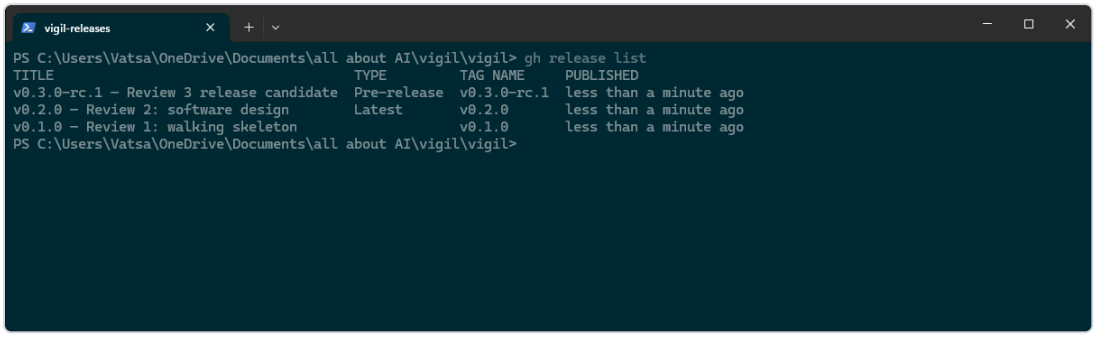


Figure 2.4 — Releases from the command line.

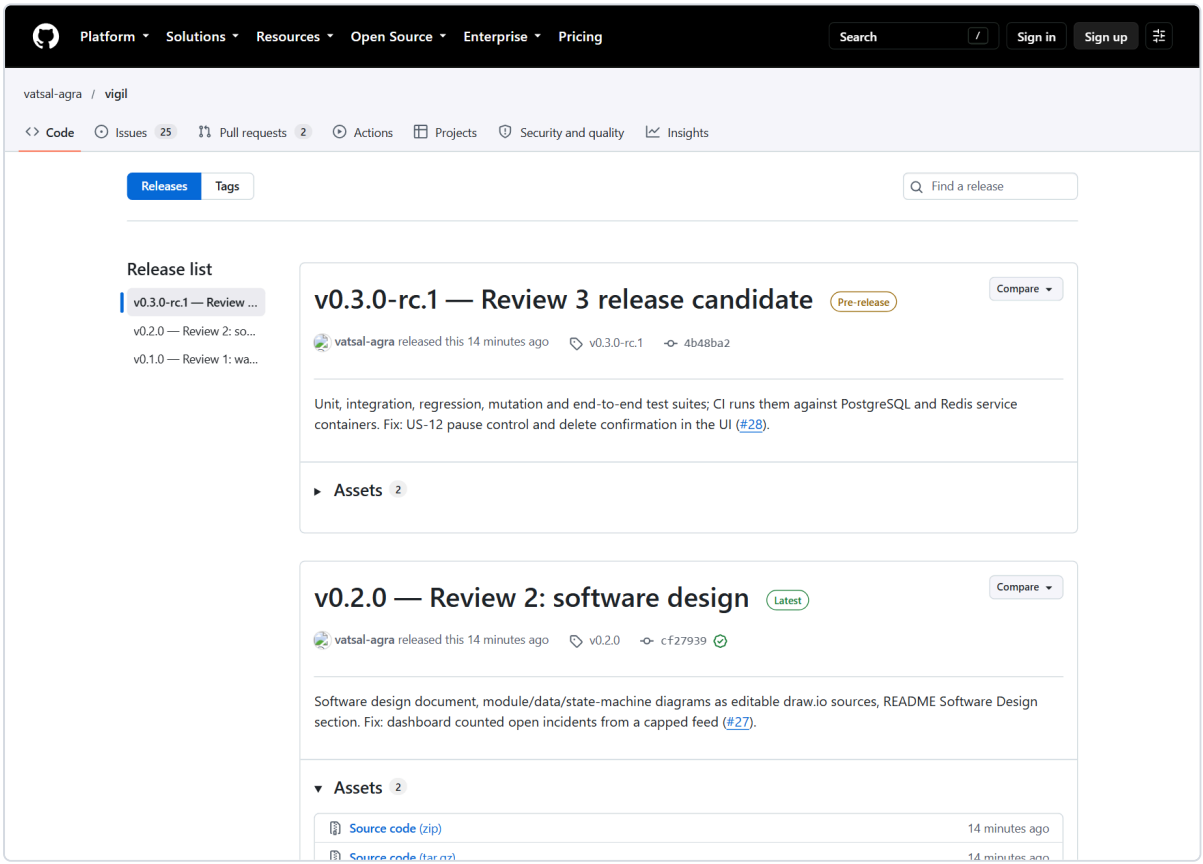


Figure 2.5 — The same releases on GitHub, with release notes and the tagged commit.

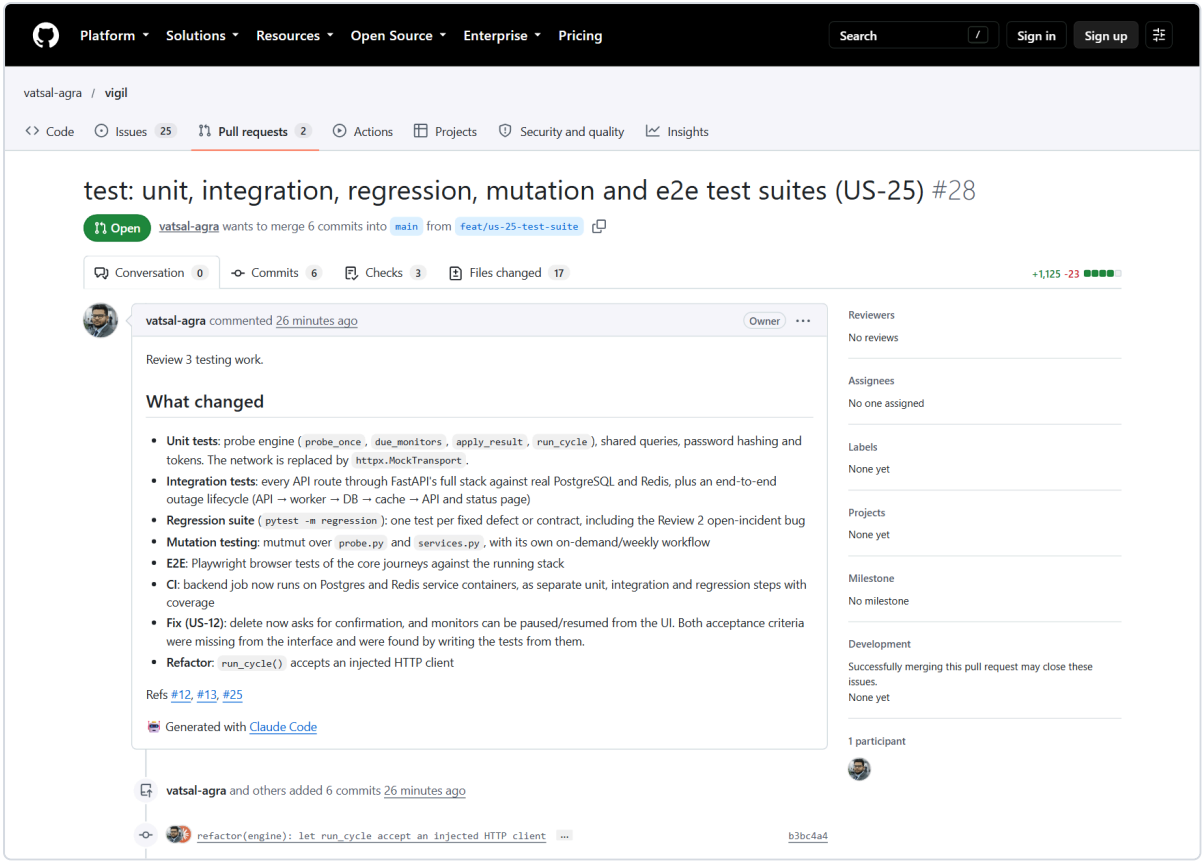


Figure 2.6 — Pull request #28 (this review's work) on GitHub.

2.2 System building

The system is built entirely from source by Docker. Both Dockerfiles are multi-stage, with named targets (`dev`, `prod`, `worker`), so one file produces the development image, the production image and, for the backend, the worker image. `docker compose build` builds all three application images, and `docker compose up` starts them with PostgreSQL and Redis in dependency order, gated on health checks.

Continuous integration builds the system again on every pull request. GitHub Actions runs the backend test suites against PostgreSQL and Redis *service containers*, type-checks and builds the Next.js frontend, and builds both production Docker images. A pull request cannot merge with a red check. Mutation testing has its own workflow, run on demand and weekly, because it is too slow to run on every change.

```
Windows PowerShell — vigil
PS C:\Users\Vatsa\OneDrive\Documents\all about AI\vigil\vigil> docker compose --progress plain build
#26 227.6   npm audit fix --force
#26 227.6
#26 227.6 Run `npm audit` for details.
#26 227.6 npm notice
#26 227.6 npm notice New major version of npm available! 10.9.9 → 12.2.0
#26 227.6 npm notice Changelog: https://github.com/npm/cli/releases/tag/v12.2.0
#26 227.6 npm notice To update run: npm install -g npm@12.2.0
#26 227.6 npm notice
#26 DONE 227.8s

#27 [web dev 3/4] COPY --from=deps /app/node_modules ./node_modules
#27 DONE 4.6s

#28 [web dev 4/4] COPY . .
#28 DONE 0.2s

#29 [web] exporting to image
#29 exporting layers
#29 exporting layers 26.7s done
#29 exporting manifest sha256:169e33be430c361128b9e1683e19bdaa537f1daa7b30b12020de61cf7aaafc5f 0.1s done
#29 exporting config sha256:caedbcce261716a6d445f9886417a81496b47d740a494de81deb502b2ebe2764 0.1s done
#29 exporting attestation manifest sha256:c6bffe351b96b7066f0f2e14b72d8b1fd8e76d535cf5914dc1cbaebf10479985 0.1s done
#29 exporting manifest list sha256:e67e28ec3b6cc64b4bddc116235444aee6e09a3b8118bdcbb465cd32431859bb
#29 exporting manifest list sha256:e67e28ec3b6cc64b4bddc116235444aee6e09a3b8118bdcbb465cd32431859bb 0.0s done
#29 naming to docker.io/library/vigil-web:latest 0.0s done
#29 unpacking to docker.io/library/vigil-web:latest
#29 unpacking to docker.io/library/vigil-web:latest 6.8s done
#29 DONE 33.8s

#30 [web] resolving provenance for metadata file
#30 DONE 0.0s
 vigil-api  Built
 vigil-worker  Built
 vigil-web  Built
```

Figure 2.7 — docker compose build: all three images built from source (last lines of output).

```
terminal — vigil
vigil $ docker compose ps
NAME             IMAGE                                                                SERVICE   STATUS                    PORTS
vigil-api-1      sha256:999d5f07fd2dd10eb37096c2469ad0069d1c6de11583df2e7caeb4e67a27021f   api       Up 38 minutes (healthy)   0.0.0.0:8000→8000/tcp,
[::]:8000→8000/tcp
vigil-cache-1    redis:7-alpine                                                       cache     Up 38 minutes (healthy)   0.0.0.0:6379→6379/tcp,
[::]:6379→6379/tcp
vigil-db-1       postgres:16-alpine                                                   db        Up 38 minutes (healthy)   0.0.0.0:5432→5432/tcp,
[::]:5432→5432/tcp
vigil-web-1      sha256:d4c59ef389e2d7fe4df1f958781c6a78a12d5b7d0e12beb329ebce8120b5eb00   web       Up 35 minutes             0.0.0.0:3000→3000/tcp,
[::]:3000→3000/tcp
vigil-worker-1   sha256:9c1d7000643c3fa9d53d25ebbf180a12d55430c4ce55d45ad5e5cf1b8d76d997   worker    Up 38 minutes             8000/tcp
```

Figure 2.8 — The built system running: five containers, with db, cache and api reporting healthy.

```
terminal — vigil
vigil $ docker images 'vigil-*'
REPOSITORY     TAG       IMAGE ID       CREATED          SIZE
vigil-api      latest    e0d66ea13996   7 minutes ago    318MB
vigil-web      latest    e67e28ec3b6c   13 minutes ago   922MB
vigil-worker   latest    01e97de8891a   34 minutes ago   318MB
```

Figure 2.9 — The images the build produced.

```
terminal — vigil
vigil $ gh run list --limit 12
completed       success test: unit, integration, regression, mutation and e2e test suites (US-25)       CI      feat/us-25-test-suite   pull_request
38717956344     3m50s   2026-10-15T10:07:04Z
completed       success test: unit, integration, regression, mutation and e2e test suites (US-25)       CI      feat/us-25-test-suite   pull_request
38715471672     4m1s    2026-10-15T09:46:57Z
completed       success Merge pull request #27 from vatsal-agra/docs/design-review-2    CI      main    push    35994798199     3m25s
2026-09-24T11:44:37Z
completed       success docs(design): Review 2 software design document, diagrams and README section    CI      docs/design-review-2    pull_request
35988366679     3m48s   2026-09-24T10:38:08Z
completed       success docs(us-16): live-environment screenshots and rebuilt submission document       CI      feat/us-16-public-status-page
pull_request    30652052472     1m39s   2026-07-31T17:40:00Z
completed       success feat: initial commit — Vigil monitoring platform        CI      main    push    30647805657     4m22s   2026-07-31T16:36:30Z
```

Figure 2.10 — CI runs for every pull request and every push to main.

```
terminal — vigil
vigil $ gh run view 38717956344 --verbose | head -n 45

✓ feat/us-25-test-suite CI vatsal-agra/vigil#28 · 38717956344
Triggered via pull_request about 6 minutes ago

JOBS
✓ Backend — tests in 51s (ID 116365152269)
  ✓ Set up job
  ✓ Initialize containers
  ✓ Run actions/checkout@v4
  ✓ Run actions/setup-python@v5
  ✓ Install dependencies
  ✓ Unit tests
  ✓ Integration tests
  ✓ Regression tests
  ✓ Coverage report
  ✓ Post Run actions/setup-python@v5
  ✓ Post Run actions/checkout@v4
  ✓ Stop containers
  ✓ Complete job
✓ Frontend — build in 36s (ID 116365152707)
  ✓ Set up job
  ✓ Run actions/checkout@v4
  ✓ Run actions/setup-node@v4
  ✓ Install dependencies
  ✓ Type-check and build
  ✓ Post Run actions/setup-node@v4
  ✓ Post Run actions/checkout@v4
  ✓ Complete job
✓ Docker — build images in 2m54s (ID 116365503954)
  ✓ Set up job
  ✓ Run actions/checkout@v4
  ✓ Run docker/setup-buildx-action@v3
  ✓ Build API image
  ✓ Build web image
  ✓ Post Build web image
  ✓ Post Build API image
  ✓ Post Run docker/setup-buildx-action@v3
  ✓ Post Run actions/checkout@v4
  ✓ Complete job

ANNOTATIONS
! Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on
Node.js 24: actions/checkout@v4, actions/setup-python@v5. For more information see:
https://github.blog/changelog/2025-09-19-deprecation-of-node-20-on-github-actions-runners/
Backend — tests: .github#2

- "The ubuntu-latest label will migrate to Ubuntu 26 beginning October 19, 2026. For more
information, see https://github.com/actions/runner-images/issues/14748"
```

Figure 2.11 — One CI run step by step: unit, integration and regression suites, coverage, frontend build, both image builds.

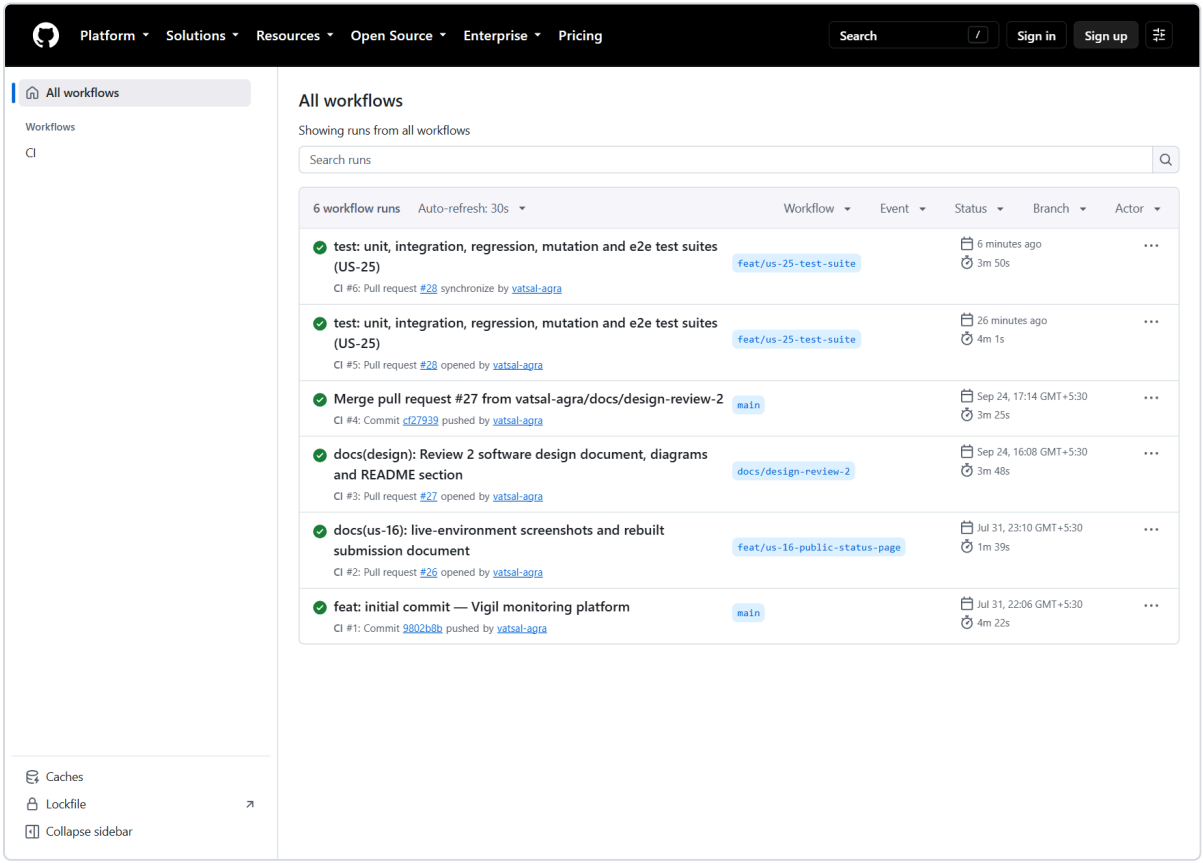
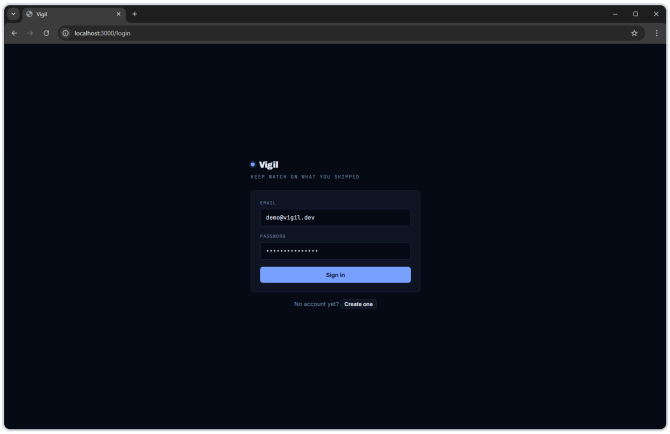


Figure 2.12 — GitHub Actions: every run of the CI workflow, all green.

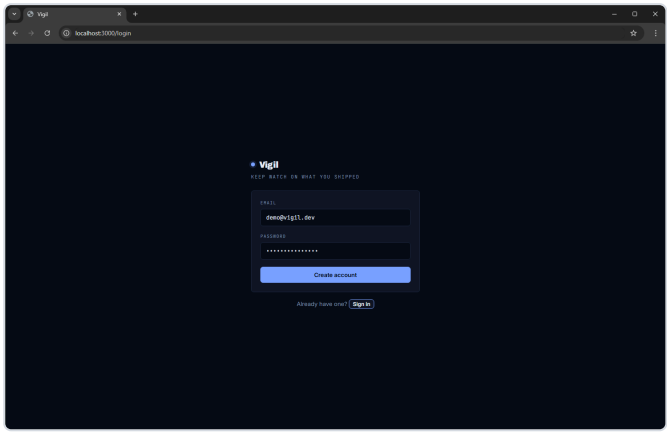
3 Developed Functionalities

Each screenshot is a browser window on the running stack, with the `localhost` address bar visible. The tag on each caption names the user stories the feature satisfies.



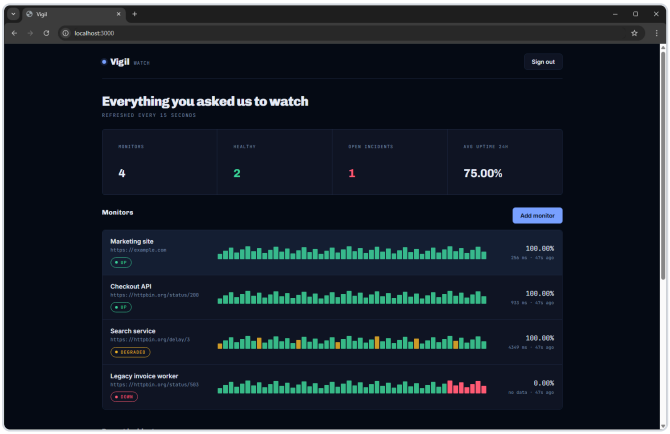
1. Sign in US-02

The entry point. A wrong password shows a message that does not reveal whether the email exists.



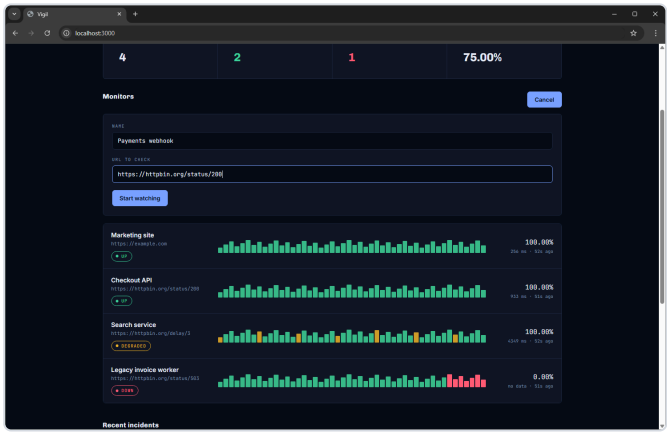
2. Create an account US-01

The same card switches to registration; an 8+ character password creates the account and signs the user in.



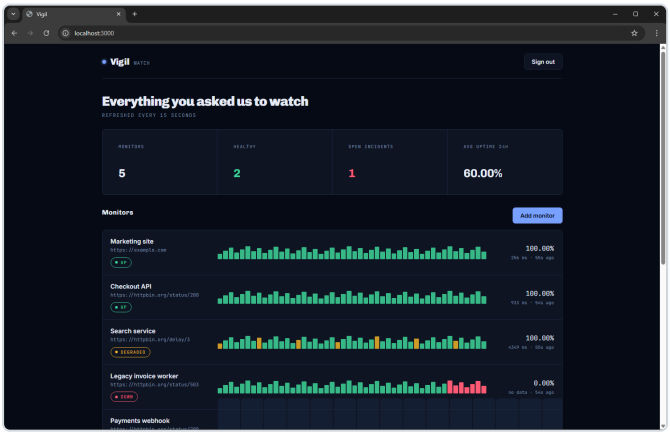
3. Dashboard US-05, 08, 09

Every monitor on one screen with status, 24-hour uptime and latency; summary tiles; refreshes every 15 seconds.



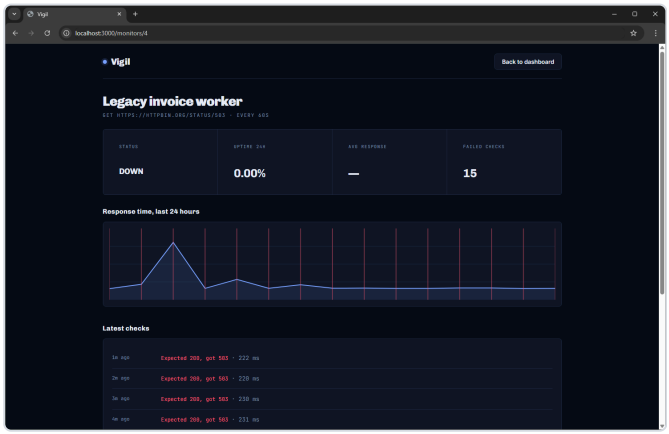
4. Add a monitor US-06, 07

Only a name and URL are required; method, expected status and interval default sensibly.



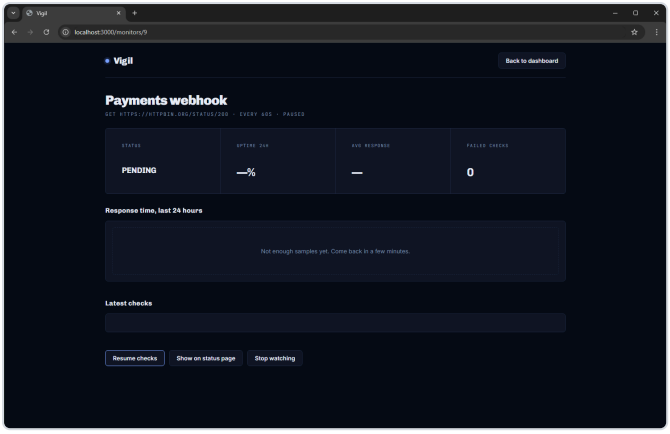
5. New monitor appears immediately US-06

The new monitor is listed at once as PENDING; the worker picks it up on its next cycle without a restart.



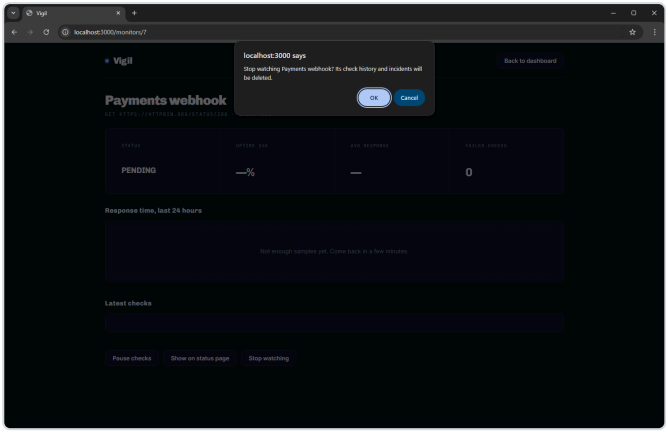
6. Monitor detail and history US-10, 11

24-hour response-time chart with failed checks as red markers, and the raw check log in plain English.



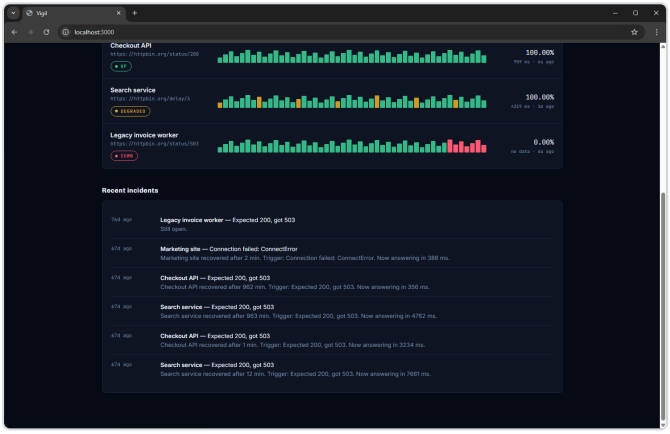
7. Pause and resume US-12

Pausing takes the monitor out of the probe schedule; the subtitle shows PAUSED and the button offers Resume.



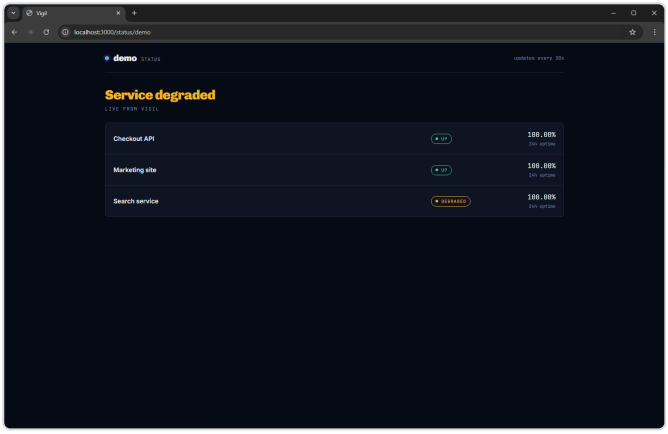
8. Delete with confirmation US-12

Stop watching asks before deleting, because history and incidents are deleted with the monitor.



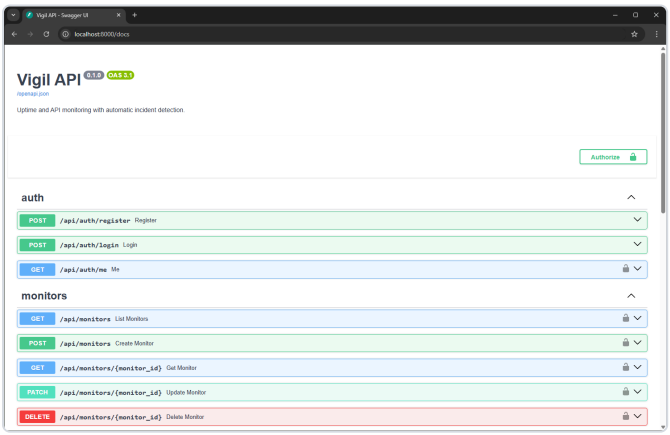
9. Incident feed US-13, 14, 15, 21

Open incidents pinned first; resolved ones carry the plain-English summary written at recovery.



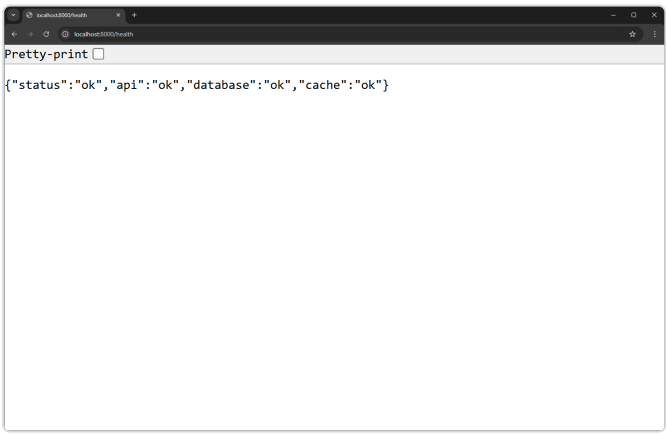
10. Public status page US-16, 17

No login. Overall banner from the worst status; only monitors marked public; no URLs or raw errors.



11. API documentation US-23

FastAPI generates interactive OpenAPI documentation for every endpoint at /docs.



12. Health endpoint US-24

GET /health reports the API, database and cache individually; Docker's health check uses it.

4 Tools and Technologies Used

Concern	Tool / technology	Used for
Frontend	**Next.js 15, React, TypeScript 5**	App Router pages, typed API client; next build fails on a type error.
Backend	**Python 3.12, FastAPI, Pydantic 2**	HTTP API, request validation, dependency injection, OpenAPI docs.
Persistence	**PostgreSQL 16, SQLAlchemy 2.0**	Relational store for users, monitors, checks and incidents; ORM models.
Cache	**Redis 7**	Cache-aside for the dashboard and public status page, invalidated on write.
Probing	**httpx**	HTTP client for probes; MockTransport for deterministic tests.
Security	**PBKDF2-SHA256, PyJWT**	Salted password hashing; signed 24-hour bearer tokens.
Containers	**Docker, Docker Compose v2**	Multi-stage images; five-service stack with health-gated start-up.
Unit / integration testing	**pytest, FastAPI TestClient**	Test runner with unit / integration / regression markers.
Coverage	**pytest-cov (coverage.py)**	Line coverage per module, reported in CI.
Mutation testing	**mutmut 2.5**	Mutates probe.py and services.py to measure how strong the tests are.
End-to-end testing	**Playwright (Chromium)**	Browser tests of the core journeys against the running stack.
Version control	**Git, GitHub, GitHub CLI**	GitHub Flow, pull requests, semantic-version tags and releases.
CI	**GitHub Actions**	Tests on Postgres/Redis service containers, frontend build, Docker image builds.
Project management	**GitHub Issues and Projects**	25 user stories as issues; board with MoSCoW, Epic and Points fields.
Diagrams	**draw.io (diagrams.net)**	Architecture, DFD, ERD and UML diagrams as editable .drawio sources.
UI design	**Figma (free plan)**	Six wireframes, committed as Figma-importable SVG.
Editor	**VS Code**	Python, Pylance, ESLint, Docker and GitLens extensions.

5 Project Demo

The demo runs from a clean clone in under two minutes, with no host dependencies beyond Docker:

```
git clone https://github.com/vatsal-agra/vigil.git && cd vigil
docker compose up --build          # five containers; demo data seeds automatically
# dashboard  http://localhost:3000   demo@vigil.dev / vigil-demo-2026
# status     http://localhost:3000/status/demo
# API docs   http://localhost:8000/docs
```

#	Show	What to point out
1	`docker compose ps`	Five containers; db, cache and api healthy. One command, no manual steps (US-23).
2	Sign in, dashboard	All monitors on one screen; status shown by word and colour; auto-refresh every 15 s.
3	Add a monitor for `https://httpbin.org/status/503`	Appears as PENDING; after two probe cycles (~1 min) it turns DOWN and an incident opens. One failure alone does not (flap guard).
4	Open the Legacy invoice worker	Red failure markers on the chart; the check log reads "Expected 200, got 503".
5	Incident feed	Open incident pinned first; resolved incidents carry a one-sentence summary.
6	Public status page in a private window	No login; only public monitors; overall banner from the worst status.
7	`docker compose stop cache`, reload the dashboard	Still answers (HTTP 200): cache errors are swallowed and reads fall through to PostgreSQL; `/health` reports `"cache": "unreachable"`. Measured honestly, each request is slow (about 8 s here), because resolving the stopped container's hostname times out before Redis is even tried. The planned fix is a circuit breaker that skips Redis for a few seconds after a failure.
8	`docker compose exec api pytest -m regression -v`	The regression suite live; then the CI checks on pull request #28.

6 Problem Statement

A developer ships a side project, a client site or an internal API. It breaks at 3 a.m., and they find out at 11 a.m. from a message that begins "hey, is your site down?" The existing options each fail this person in a specific way:

Option	Why it does not fit
Pingdom, Better Uptime, Datadog	Priced for companies. Free tiers cap at a handful of monitors and put the status page behind a paid plan.
UptimeRobot free tier	Five-minute interval and days of retention: you learn about the outage after it has ended.
A cron job running curl	No history, no trend, no incident record, nothing to show a client.
Uptime Kuma (self-hosted)	The closest comparison, but a single process on SQLite: no separate worker, no cache tier, no plain-language incident narrative.

> **Problem statement.** Developers and small teams who run three to twenty web services need to learn about failures before their users do, and to explain those failures afterwards, without paying per monitor or building monitoring themselves. **Vigil** is a self-hosted platform that probes each service on a schedule, opens an incident on its own after repeated failures, closes it on recovery with a plain-English explanation, and publishes a public status page, all started with one `docker compose up`.

Users. Rhea, a solo founder (primary): she needs to know before her customers do. Arjun, a backend engineer (secondary): he needs latency history and alerts he can trust. Meera, a freelancer (tertiary): she needs unlimited monitors and an uptime figure per client. **Success** is measured by time to first monitor (under 90 s), detection latency (within twice the interval), false-incident rate (under one per monitor per week) and dashboard p95 (under 200 ms at 50 monitors).

7 User Stories

Twenty-five stories across six epics, each in role / capability / benefit form, with three acceptance criteria and a story-point estimate. They exist as GitHub Issues (github.com/vatsal-agra/vigil/issues) on a Project board with MoSCoW, Epic and Points fields, all generated from one file, `scripts/stories.sh`. **Must** 16 stories, 58 pts · **Should** 6 stories, 22 pts · **Could** 2 stories, 6 pts · **Won't** 1 stories, 5 pts.

Story	User story	Acceptance criteria
US-01 Must 3 pts	**Create an account** As a developer running side projects, I want to sign up with an email and password so that my monitors are private to me.	• A valid email and an 8+ character password creates the account and signs me in • A duplicate email returns 409 with 'That email already has an account.' • Passwords are stored as salted PBKDF2-SHA256 hashes
US-02 Must 3 pts	**Sign in** As a returning user, I want to sign in so that I can reach my dashboard.	• Correct credentials return a JWT valid for 24 hours • Wrong credentials return 401 without revealing whether the email exists • The token is sent as a Bearer header on every API call
US-03 Must 2 pts	**Stay signed in, and sign out on demand** As a user, I want my session to survive a refresh and a sign-out button that ends it so that I am not retyping my password all day.	• A page refresh keeps me signed in until the token expires • Sign out clears the token and redirects to /login • An expired or tampered token redirects to /login rather than showing a blank screen
US-04 Must 3 pts	**Never see another account's data** As a user, I want hard isolation between accounts so that self-hosting for a small team is safe.	• Every monitor and incident query filters by owner_id • Requesting a monitor I do not own returns 404, not 403 • A request with no Authorization header returns 401
US-05 Must 5 pts	**See everything I watch on one screen** As a developer, I want a single dashboard listing every monitor so that I know whether anything is broken in under five seconds.	• Each row shows name, URL, status, 24h uptime, average response time, and last-checked time • Status is conveyed by label as well as colour • Zero monitors shows an empty state with a path to add one
US-06 Must 3 pts	**Add a monitor in under ten seconds** As a developer, I want to add a URL with just a name so that setup is not a chore.	• Only name and URL are required; method, expected status, and interval have defaults • The monitor appears immediately with status pending • The worker picks it up next cycle without a restart
US-07 Must 3 pts	**Configure how a check is made** As a backend developer, I want to set method, expected status, and interval so that I can monitor API health endpoints, not just web pages.	• Method, expected status, and interval are editable on create and edit • Interval is clamped server-side to 30-3600s with a readable 422 • A response status differing from expected counts as a failure
US-08 Must 3 pts	**Judge health from the list, not the detail page** As a user, I want uptime and average response time on each row so that I can spot a service that is up but getting slower.	• Uptime is successful checks over total checks in the trailing 24h, to two decimals • Average latency uses successful checks only • A monitor with no checks shows an em dash, not 0%
US-09 Should 2 pts	**Watch the dashboard update itself** As a user with the dashboard on a second screen, I want it to refresh itself so that it works as an ambient status display.	• The dashboard re-fetches every 15 seconds • The poll is cancelled on unmount so timers do not leak • A failed poll shows an inline error and keeps retrying
US-10 Must 5 pts	**Read the response-time history of one service** As a developer investigating slowness, I want a 24-hour response-time chart so that I can tell a spike from a trend.	• The chart plots every successful check in the window, oldest to newest • Failed checks are drawn as vertical markers so gaps are explained • Fewer than two samples shows 'Not enough samples yet'
US-11 Must 2 pts	**Read the raw check log** As a developer, I want the last ten checks with status code, latency, and error so that I can see exactly what the server returned.	• Each row shows relative time, status or error, and latency • Errors are phrased for humans, not as exception class names • Successes and failures are visually distinguishable

US-12 Must 3 pts	**Pause or remove a monitor** As a user decommissioning a service, I want to pause or delete a monitor so that a retired service stops paging me.	• Pausing sets is_active false and the worker skips it • Deleting cascades to checks and incidents • Deletion asks for confirmation first
US-13 Must 8 pts	**Have outages detected without me watching** As a developer asleep at 3am, I want the system to notice failures on its own so that monitoring is not something I have to remember.	• A dedicated worker container probes every active monitor whose interval has elapsed • Two consecutive failures flip the monitor to down and open an incident • A single failed check does not open an incident
US-14 Must 5 pts	**Have recoveries detected too** As a user, I want incidents to close themselves so that the incident list reflects reality without manual filing.	• The first success after a failure resolves the open incident • The incident gets a plain-English summary with cause, duration, and current latency • The failure counter resets on success
US-15 Must 3 pts	**See the incident history** As a team lead, I want a reverse-chronological incident feed so that I can answer 'how often does this break?'	• The feed shows monitor, cause, start time, duration, and state • I can filter to open incidents only • Results are capped at 100 per request
US-16 Should 5 pts	**Publish a status page my users can read** As a solo founder, I want a public page showing which services are healthy so that customers stop emailing me during an outage.	• /status/<handle> is readable with no authentication • An overall banner is derived from the worst individual status • Monitor URLs and internal errors are never exposed
US-17 Should 3 pts	**Choose what is public** As a user, I want a per-monitor visibility toggle so that internal services stay internal.	• Monitors default to private • Toggling updates the public page within one cache TTL • A page with no public monitors renders an empty state, not an error
US-18 Should 2 pts	**Acknowledge an incident** As an on-call developer, I want to acknowledge an incident so that teammates know someone is on it.	• Acknowledging is one click from the incident feed • The state persists and is visible to everyone on the account • Acknowledging does not resolve the incident
US-19 Should 5 pts	**Get an email when something breaks** As a developer, I want an email when an incident opens so that I do not need the dashboard open to find out.	• Email is sent on incident open and close, never per failed check • SMTP config comes from env vars and the feature disables cleanly when unset • A failed send retries twice then logs, never crashing the worker
US-20 Could 3 pts	**Get alerts in Slack** As a small team, we want incidents posted to Slack so that alerts land where we already talk.	• A webhook URL is configurable per account • The message names the service, the cause, and links to the monitor • Slack being down does not affect probing
US-21 Could 3 pts	**Read a plain-English explanation of an outage** As a non-technical founder, I want the incident described in a sentence I understand so that I can decide whether to tell customers.	• Every resolved incident carries a summary with service, duration, trigger, and current latency • The summary is generated at resolution and stored • No jargon in user-facing error text
US-22 Won't 5 pts	**Quiet hours** As a developer who needs sleep, I want to suppress non-critical alerts overnight so that a flaky staging box does not wake me.	• A per-account quiet window with timezone is configurable • Suppressed alerts still record incidents and appear in a morning digest • Monitors flagged critical bypass quiet hours
US-23 Must 5 pts	**Run the whole stack with one command** As a new contributor, I want docker compose up to give me a working system so that I am productive on day one.	• One command starts db, cache, api, worker, and web with no manual steps • The API waits for Postgres health before starting • Demo data seeds on an empty database
US-24 Must 2 pts	**Check whether Vigil itself is healthy** As an operator, I want a health endpoint reporting its own dependencies so that Docker and my proxy can make routing decisions.	• GET /health reports api, database, and cache individually • It returns 200 with status degraded when a dependency is down • The container HEALTHCHECK uses it

US-25 Should 5 pts	**Ship changes without breaking main** As a maintainer, I want CI to run tests and build images on every PR so that main stays deployable.	• Every PR runs pytest, next build, and docker build for both services • A failing job blocks the merge • Merging to main publishes tagged images

8 System Architecture and System Design

8.1 Architecture

Vigil is a **client–server system with a layered backend**, packaged as five containers: the Next.js frontend, the FastAPI API, a probe worker, PostgreSQL and Redis. It is deliberately not microservices. The API is one process over one database, and the only thing split out is the worker, for a concrete reason: probing is slow, blocking I/O, and an outage must never slow down the dashboard that reports it.

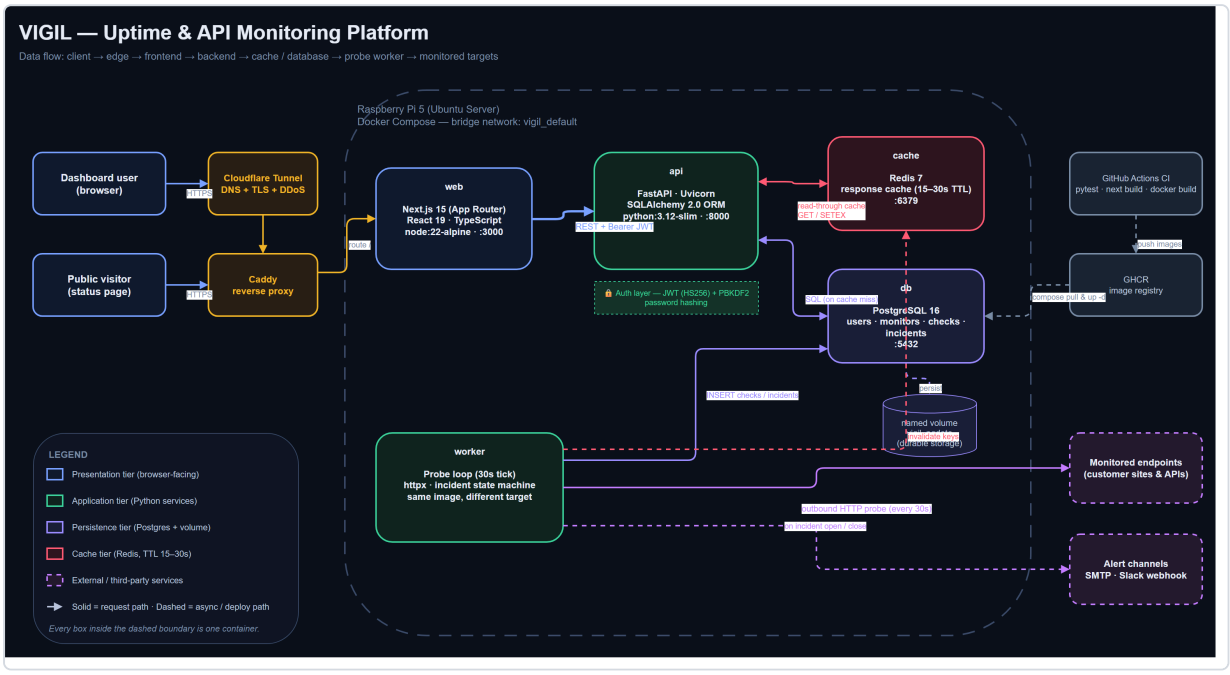


Figure 8.1 — System architecture (deployment view). Editable source: docs/design/diagrams/01-architecture.drawio

8.2 Module design

Every source file sits in exactly one layer, and dependencies only point downward. The frontend talks to the backend only through `lib/api.ts`. Routers stay thin: validate, authorise, delegate. Domain modules (`probe`, `services`, `cache`, `security`) are shared by the API and the worker, which never call each other. They meet only at the database and at a cache-key naming convention.

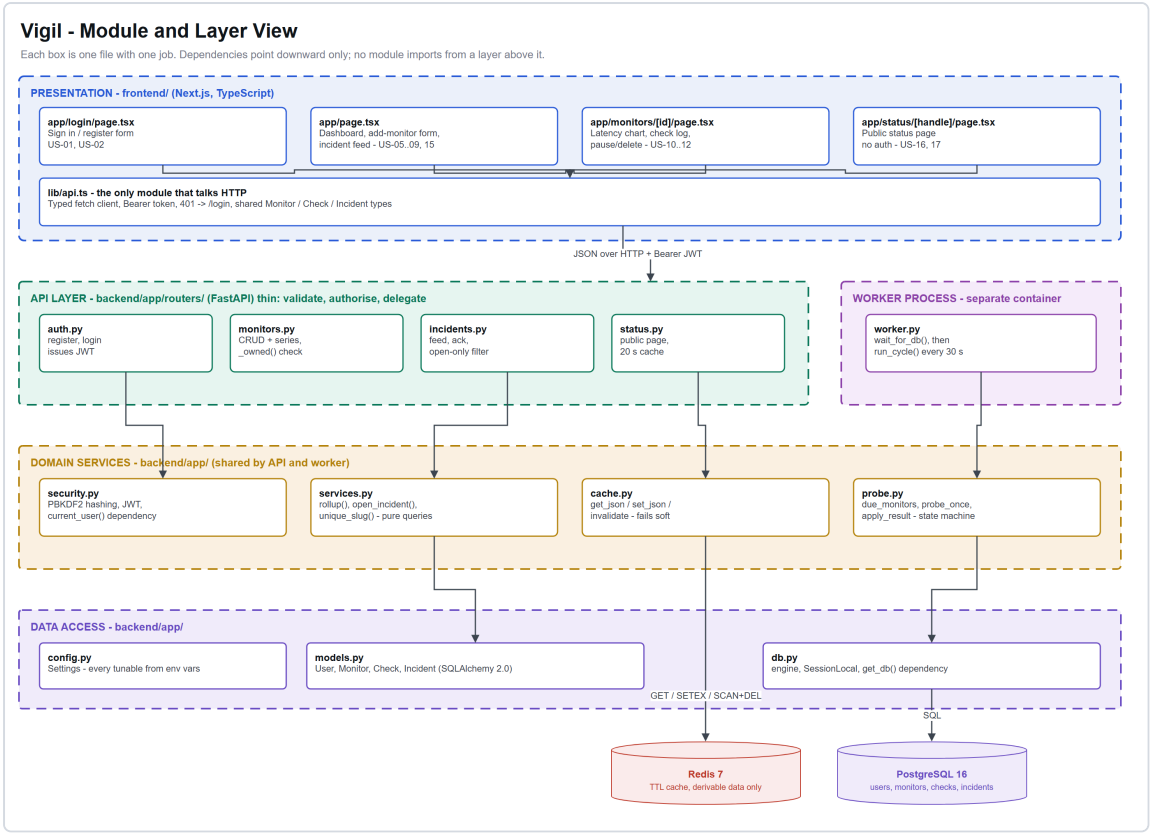


Figure 8.2 — Module and layer view (component diagram). Editable source: docs/design/diagrams/02-module-layers.drawio

8.3 Design principles and key decisions

Principle / decision	How it shows up, and why
Abstraction	Pages call `api<T>()` and never touch fetch, tokens or 401 handling. Routers call `cache.get_json()` and never touch Redis, which fails soft, so to callers the cache is optional.
High cohesion	`probe.py` holds only the checking engine, in three single-purpose steps: `due_monitors` (which), `probe_once` (measure, no database writes), `apply_result` (state transition).
Low coupling	API and worker share only the schema and cache-key names; the frontend and backend share only a JSON contract (Pydantic ↔ TypeScript types); configuration is one `Settings` object read from the environment.
Separate worker	A timing-out probe holds a thread for 8 s; outside the API, an outage cannot slow the dashboard.
Cache-aside, invalidate on write	The status page spikes when something is failing; a 20 s TTL caps the database load, and invalidation keeps data no staler than one probe cycle.
Flap guard	An incident opens on the second consecutive failure, not the first, so the alert channel stays worth reading.
404, not 403	Another user's monitor returns 404, so its existence is not leaked; enforced in one helper, `_owned()`.

The data model (Appendix C), the monitor state machine and the UML views (Appendix D) complete the design; Review 2's Software Design Document covers each decision in more depth (`docs/design/`).

9 Design of Tests

9.1 Strategy

Tests are designed from the acceptance criteria first and the code second. Every acceptance criterion that describes behaviour has at least one test, and the test name reads as the criterion. The pyramid is deliberately wide at the bottom: fast unit tests pin exact behaviour, integration tests prove that the parts work together on the real database and cache, and a handful of end-to-end tests cover the journeys a user actually takes. Mutation testing then measures whether the tests are strong enough, and the regression suite stops a fixed defect from returning.

Aspect	Decision
Environment	Real PostgreSQL 16 and Redis 7 (Docker locally, service containers in CI), isolated test database and Redis db 15; each test starts from empty tables and an empty cache.
Test doubles	Only the monitored targets are simulated (`httpx.MockTransport`), plus a fake clock where timing is asserted. The database, cache, auth and HTTP layer are real.
Design techniques	**Equivalence partitioning** (valid / missing / forged tokens), **boundary-value analysis** (interval 29/30/3600/3601 s, latency 2000.0/2000.1 ms, the exact due moment), **state-transition testing** (Appendix D, Figure D.5), **error guessing** (timeouts, refused connections, user enumeration), and **defect-based** regression tests.
Entry / exit criteria	A pull request may merge only when the unit, integration and regression suites, the frontend build and both image builds pass in CI. The mutation score is tracked over time.
Traceability	Test names cite the behaviour; regression tests cite the defect or story; the table below maps test cases to stories.

9.2 Test cases

ID	Level	Story	Technique	Input / scenario	Expected result	Automated by
TC-01	Integration	US-01	Positive	Register with a valid email and an 8+ char password	201, token returned, /me returns the user	`test_register_signs_the_user_in`
TC-02	Integration	US-01	Negative	Register an email that already exists	409 'That email already has an account.'	`test_duplicate_email_is_a_409_with_a_readable_message`
TC-03	Integration	US-01	Boundary	Password of 5 characters	422	`test_short_passwords_are_rejected`
TC-04	Integration	US-02	Error guessing	Wrong password vs unknown email	Identical 401 bodies (no enumeration)	`test_login_does_not_reveal_whether_an_email_exists`
TC-05	Integration	US-04	Equivalence	No token / forged token	401 on a protected route	`test_protected_routes_require_a_valid_token`
TC-06	Integration	US-04	Negative	User B reads, edits, deletes user A's monitor	404 for every verb; A unaffected	`test_another_users_monitor_is_a_404_for_every_verb`
TC-07	Integration	US-06	Positive	Create a monitor with only name and URL	GET / 200 / 60 s defaults, status pending	`test_a_new_monitor_needs_only_a_name_and_url`
TC-08	Integration	US-07	Boundary value	interval = 29, 30, 3600, 3601	422, 201, 201, 422	`test_interval_is_clamped_to_30_through_3600_seconds`
TC-09	Integration	US-08	Positive	One passed and one failed check	uptime 50.0, avg latency of successes only	`test_the_list_carries_uptime_and_latency_per_monitor`
TC-10	Integration	US-10	Positive	Checks inserted out of order	Series returned oldest first; hours=0 → 422	`test_the_series_endpoint_returns_checks_oldest_first`
TC-11	Integration	US-12	State	Pause a monitor	Removed from the probe schedule	`test_pausing_a_monitor_takes_it_out_of_the_probe_schedule`

TC-12	Integration	US-12	Positive	Delete a monitor with checks and incidents	204; checks and incidents deleted	`test_deleting_a_monitor_cascades_to_its_checks_and_incidents`
TC-13	Integration	US-15, 18	Positive	Filter open incidents; acknowledge	Only the open one; acknowledged but not resolved; other user 404	`test_the_incident_feed_filters_and_acknowledges`
TC-14	Integration	US-16, 17	Positive	One public and one private monitor	Status page lists only the public one, no auth	`test_the_status_page_shows_only_public_monitors_without_auth`
TC-15	Integration	US-24	Positive	GET /health	api, database, cache all ok	`test_health_reports_each_dependency`
TC-16	Integration	US-13, 14, 16	State transition	Target fails twice, then recovers	pending → down (incident) → up (resolved, summary); banner follows	`test_an_outage_is_detected_recorded_and_explained_end_to_end`
TC-17	Unit	US-13	State transition	One failure	Counter 1, status unchanged, no incident	`test_one_failure_is_noise_not_an_incident`
TC-18	Unit	US-13	State transition	Second consecutive failure	down; one incident with the cause	`test_the_second_consecutive_failure_opens_an_incident`
TC-19	Unit	US-14	State transition	Success after an outage	Incident resolved; summary names cause and minutes	`test_recovery_resolves_the_incident_and_writes_a_summary`
TC-20	Unit	US-08	Boundary value	Latency 2000.0 vs 2000.1 ms	up vs degraded	`test_slow_but_successful_checks_are_degraded_above_two_seconds`
TC-21	Unit	US-11	Error guessing	Timeout; refused connection	Readable error text, never an exception name	`test_probe_once_turns_a_timeout_into_a_readable_failure`
TC-22	Unit	US-13	Boundary value	Checked exactly one interval ago (frozen clock)	Due	`test_a_monitor_is_due_at_exactly_its_interval`
TC-23	Unit	—	Positive	Worker result for one owner	Only that owner's cache keys invalidated	`test_a_result_invalidates_exactly_the_affected_cache_keys`
TC-24	Regression	Review 2 defect	Defect-based	Open incident older than 100 newer ones	Still returned by ?open_only=true	`test_R1_open_incident_beyond_the_feed_cap_is_still_reported`
TC-25	Regression	US-16	Defect-based	Public page for a monitor with an internal URL	No URL or error text in the response	`test_R5_the_public_page_never_leaks_urls_or_raw_errors`
TC-26	E2E	US-03	Use case	Visit / while signed out	Redirected to /login	`test_signed_out_visitors_are_sent_to_sign_in`
TC-27	E2E	US-06, 12	Use case	Add, pause, resume, cancel delete, confirm delete	Each step visible in the UI; monitor gone at the end	`test_monitor_lifecycle_add_pause_resume_and_confirmed_delete`
TC-28	E2E	US-16	Use case	Open /status/demo without signing in	Public monitors shown; private one hidden	`test_the_public_status_page_needs_no_login`

9.3 Results

All 79 backend tests and all 6 end-to-end tests pass locally and in CI. Line coverage is 92%, with `probe.py` and `services.py` at 100%. The mutation score over those two modules rose from 79.2% to 98.3% after the mutation-driven tests, and to 99.2% once the last weak test was fixed; the only survivor is an equivalent mutant (Section 1.3). Writing tests from the acceptance criteria found two genuine defects, both now fixed and guarded: the open-incident count (Review 2, R1) and the missing pause control and delete confirmation (US-12, E2E).

Appendix A — Software Requirements Specification

A.1 Introduction

Purpose. This specification states what Vigil must do and how well, for the developer and the reviewers. **Scope.** Vigil is a self-hosted HTTP/HTTPS uptime and API monitor with automatic incident detection, plain-language incident summaries and a public status page. Out of scope for this release: log aggregation, APM, multi-region probing, mobile apps and billing. **Definitions.** *Monitor*: a URL checked on an interval. *Check*: one probe result. *Incident*: an outage, open from the second consecutive failure until the next success. *Handle*: the public name of an account's status page.

A.2 Overall description

Product perspective. A standalone system of five containers (Section 8). **User classes.** Registered users (developers and operators) and anonymous public visitors. **Operating environment.** Docker Engine 24+ with Compose v2 on Linux, macOS, Windows or a Raspberry Pi 5; any modern browser. **Constraints.** One developer, one semester; zero recurring cost; every image slim. **Assumptions.** The host has stable outbound network access; users own the endpoints they monitor; HTTP checks cover the need for this release.

A.3 Functional requirements

ID	Requirement	Stories
FR-01	The system shall let a visitor register with an email and a password of at least 8 characters, and sign them in.	US-01
FR-02	The system shall authenticate users by email and password and issue a bearer token valid for 24 hours.	US-02, 03
FR-03	The system shall restrict every monitor, check and incident to its owner and answer 404 for another owner's resource.	US-04
FR-04	The system shall let a user create a monitor from a name and URL, with optional method, expected status and interval (30–3600 s).	US-06, 07
FR-05	The system shall list a user's monitors with status, 24-hour uptime, average latency and last-checked time.	US-05, 08
FR-06	The system shall let a user pause, resume, publish, unpublish and (after confirmation) delete a monitor.	US-12, 17
FR-07	The system shall probe every active monitor whose interval has elapsed and record status code, latency and error.	US-13
FR-08	The system shall open an incident after two consecutive failed checks and never after one.	US-13
FR-09	The system shall resolve an open incident on the first successful check and store a plain-English summary.	US-14, 21
FR-10	The system shall mark a monitor degraded when a successful check takes longer than 2000 ms.	US-08
FR-11	The system shall provide an incident feed, filterable to open incidents, with acknowledgement.	US-15, 18
FR-12	The system shall serve an unauthenticated status page per account showing only public monitors and an overall banner.	US-16, 17
FR-13	The system shall show a 24-hour response-time history and the latest checks for each monitor.	US-10, 11
FR-14	The system shall report the health of the API, database and cache individually.	US-24

A.4 Non-functional requirements

ID	Quality	Requirement

NFR-01	Performance	Dashboard list p95 under 200 ms at 50 monitors; public status page served from cache (20 s TTL).
NFR-02	Reliability	An outage of a monitored target, or of Redis, must not affect the API's availability; the worker survives a failed cycle.
NFR-03	Detection	An incident opens within twice the configured interval of the first real failure.
NFR-04	Security	Salted PBKDF2-SHA256 password hashes; signed JWTs; no resource existence leaked across accounts; secrets from the environment only.
NFR-05	Usability	Status conveyed by word as well as colour; errors in plain English; a monitor needs only two fields.
NFR-06	Maintainability	Layered modules with downward-only dependencies; CI blocks merges on failing tests; mutation score tracked.
NFR-07	Portability	Runs anywhere Docker runs, including a Raspberry Pi 5; no host dependencies besides Docker.
NFR-08	Deployability	One command (docker compose up) starts a working, seeded system.

A.5 External interfaces

User interface: six screens (sign in / register, dashboard, add monitor, monitor detail, incident feed, public status page), shown in Section 3. **Software interface:** a JSON REST API (OpenAPI documentation at `/docs`):

Method	Path	Purpose
`POST`	`/api/auth/register`	Create account, return token
`POST`	`/api/auth/login`	Sign in, return token
`GET`	`/api/auth/me`	Current user
`GET`	`/api/monitors`	List with 24 h rollup (cached 15 s)
`POST`	`/api/monitors`	Create monitor
`GET`	`/api/monitors/{id}`	One monitor with rollup
`PATCH`	`/api/monitors/{id}`	Update, pause, publish
`DELETE`	`/api/monitors/{id}`	Delete with history
`GET`	`/api/monitors/{id}/checks`	Response-time series (cached 30 s)
`GET`	`/api/incidents`	Feed; ?open_only=true
`POST`	`/api/incidents/{id}/acknowledge`	Acknowledge
`GET`	`/api/status/{handle}`	Public status page (no auth, cached 20 s)
`GET`	`/health`	Dependency health

Communications: HTTPS from clients through the reverse proxy in production; HTTP(S) probes from the worker to monitored endpoints; PostgreSQL and Redis protocols inside the Compose network only.

Appendix B — Data Flow Diagrams

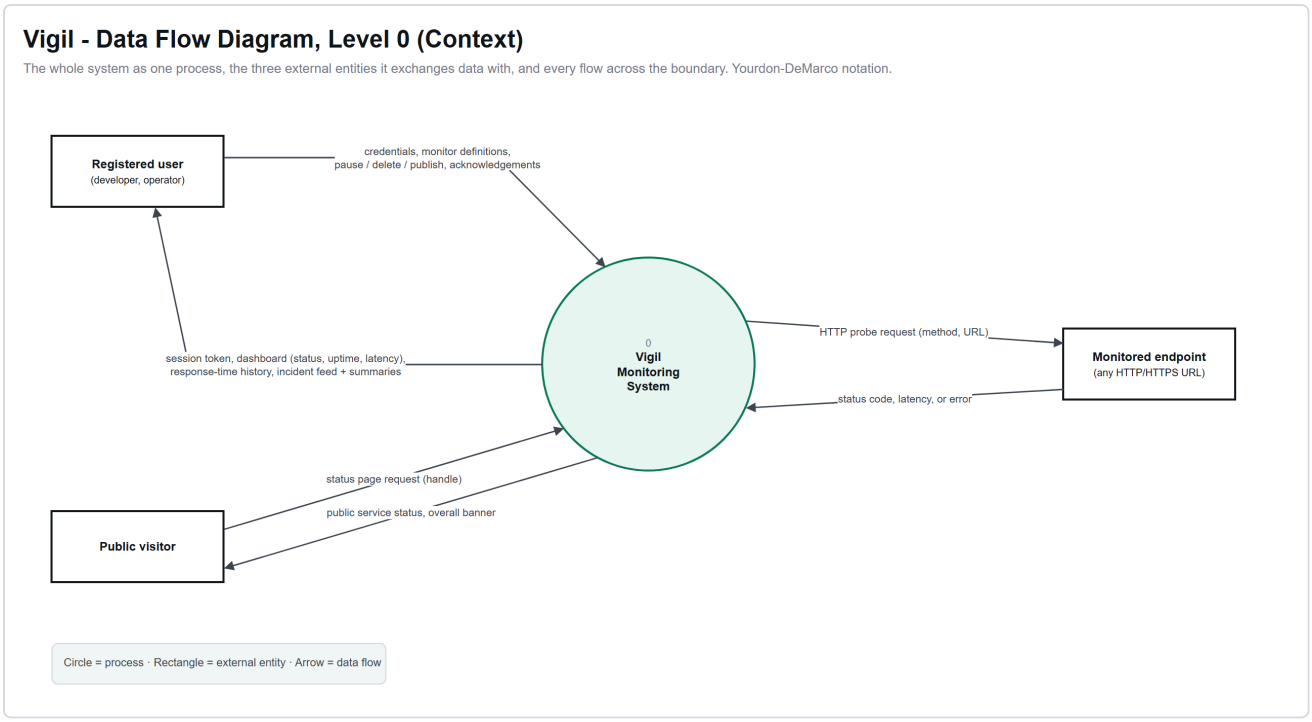


Figure B.1 — DFD Level 0 (context diagram). Editable source: docs/design/diagrams/05-dfd-level-0.drawio

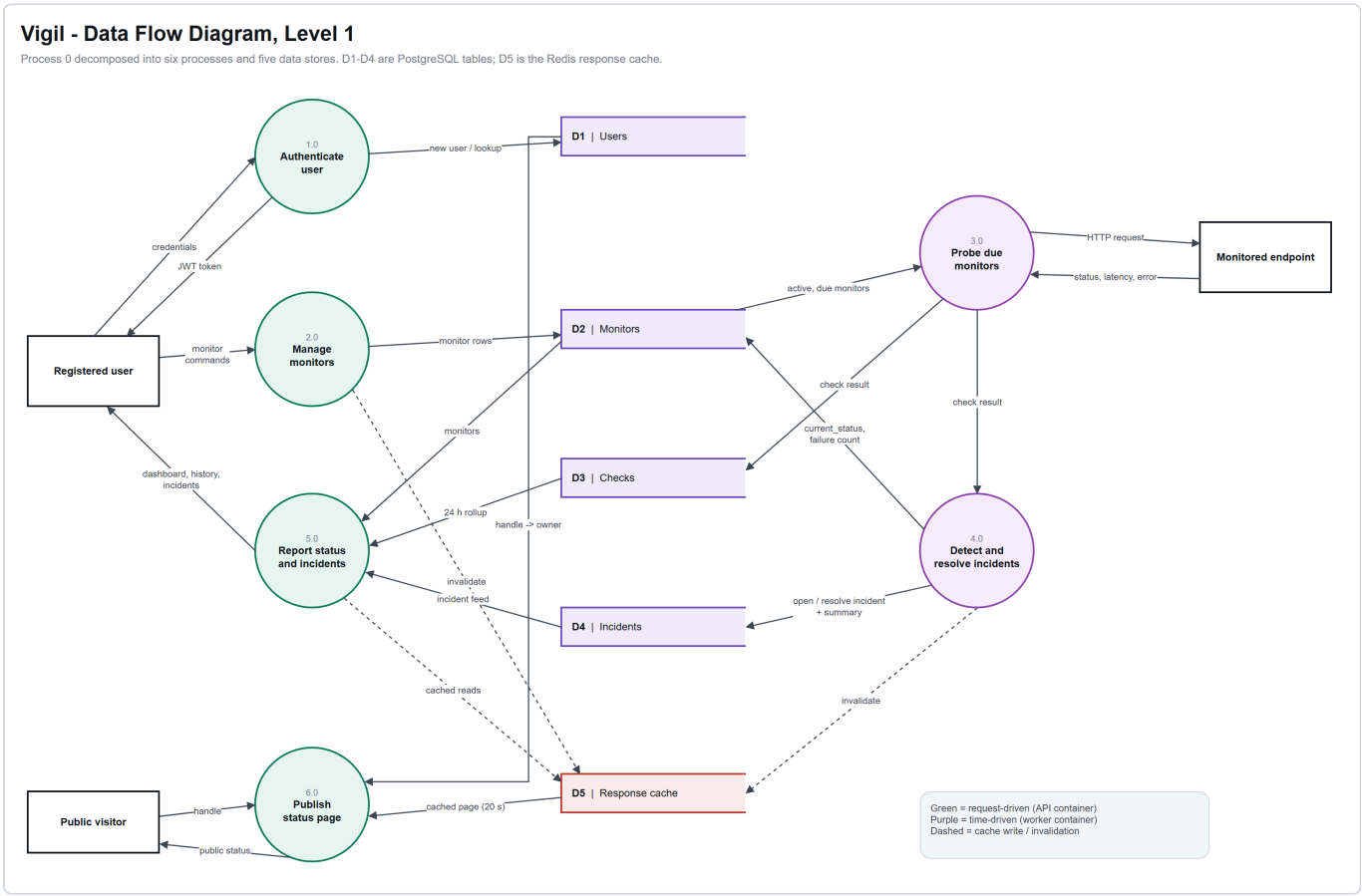


Figure B.2 — DFD Level 1: six processes and five data stores. Editable source: docs/design/diagrams/06-dfd-level-1.drawio

Appendix C — Entity-Relationship Diagram

Crow's-foot notation. A user owns zero or more monitors; a monitor records zero or more checks and opens zero or more incidents. Checks and incidents are deleted with their monitor (composition). An incident is open exactly when `resolved_at` is null, so there is no second status column to drift out of sync.

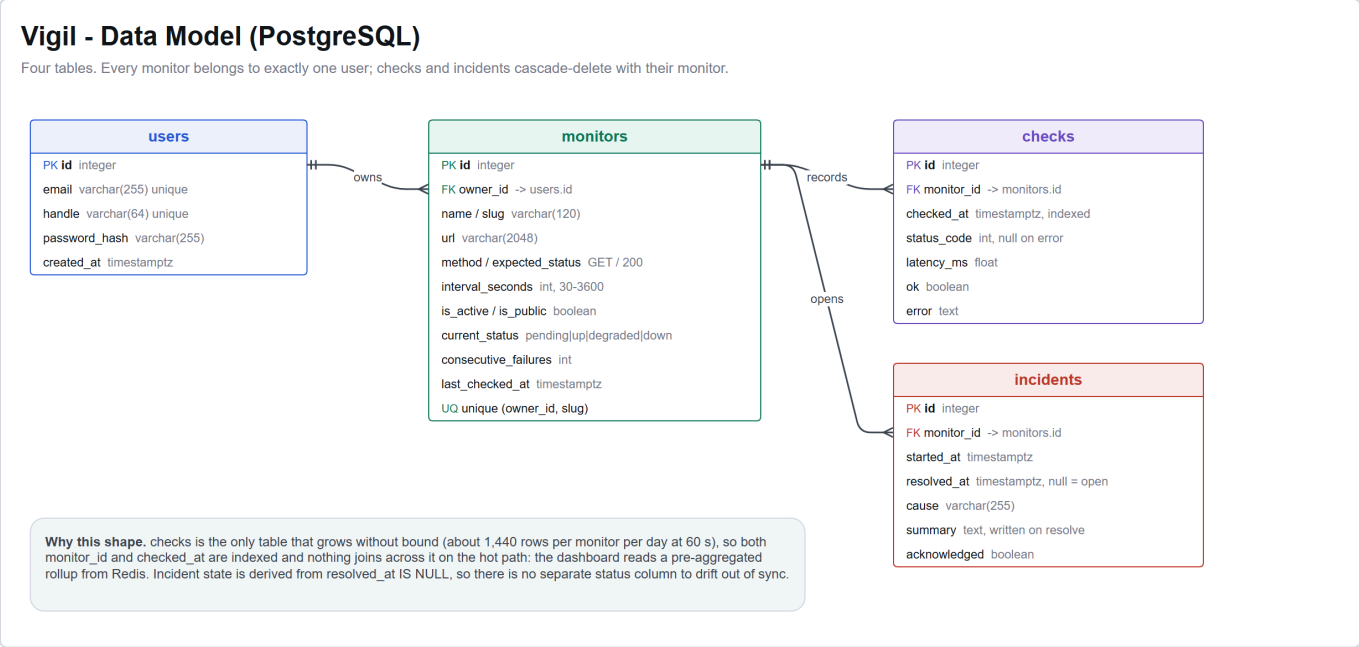


Figure C.1 — Entity-relationship diagram (PostgreSQL schema). Editable source: docs/design/diagrams/03-data-model.drawio

Appendix D — UML Diagrams

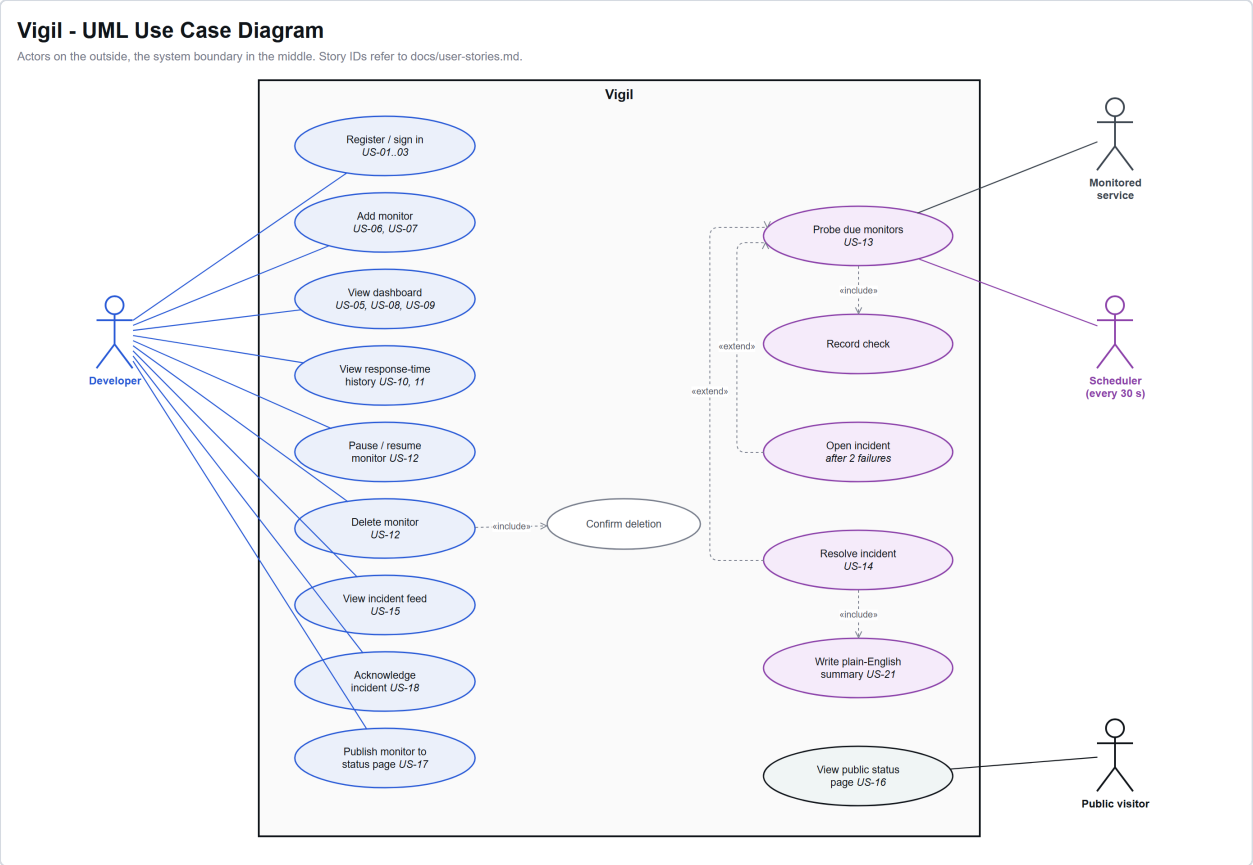


Figure D.1 — Use case diagram. Editable source: docs/design/diagrams/07-use-case.drawio

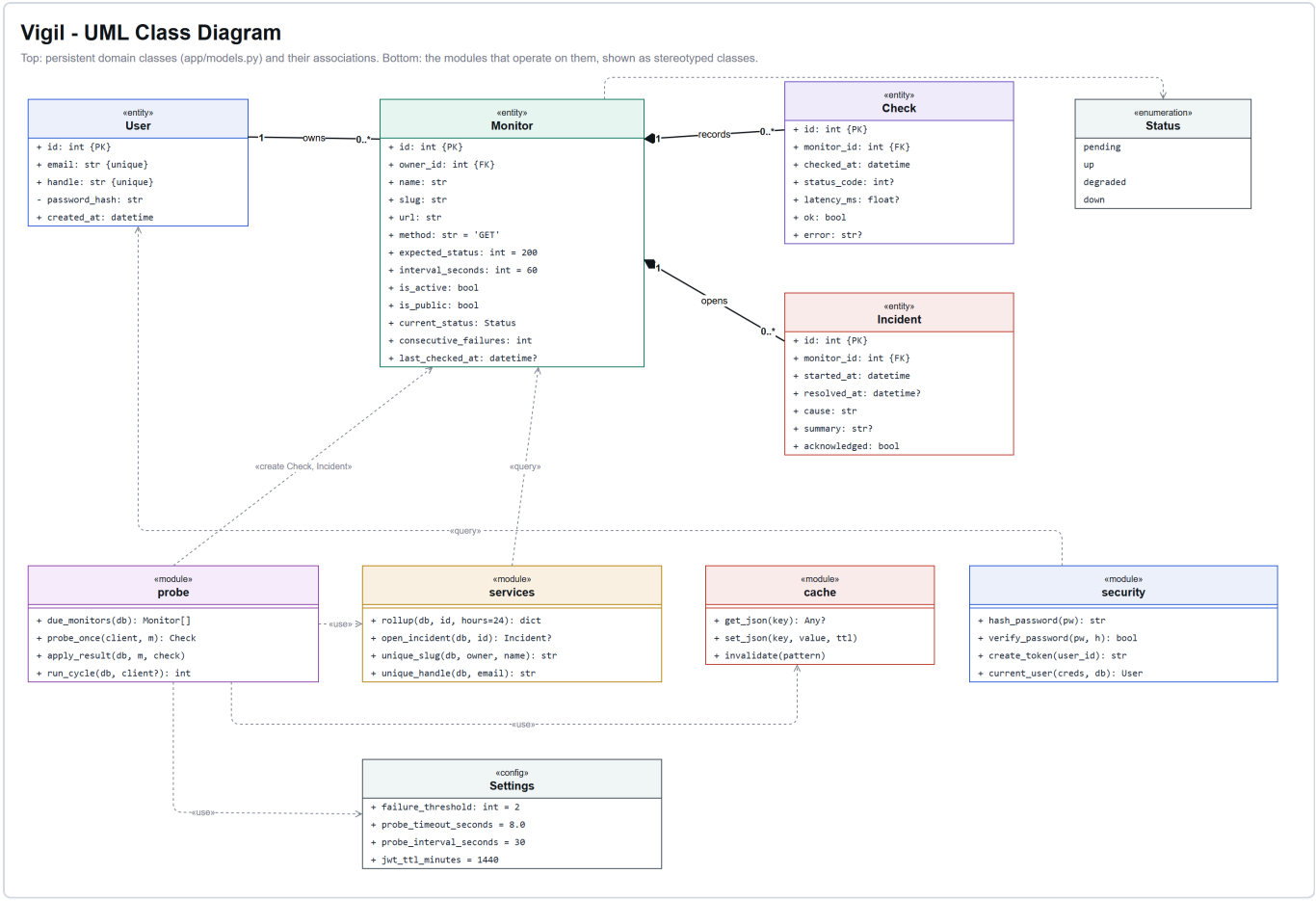


Figure D.2 — Class diagram: domain entities and the modules that operate on them. Editable source: docs/design/diagrams/08-class.drawio

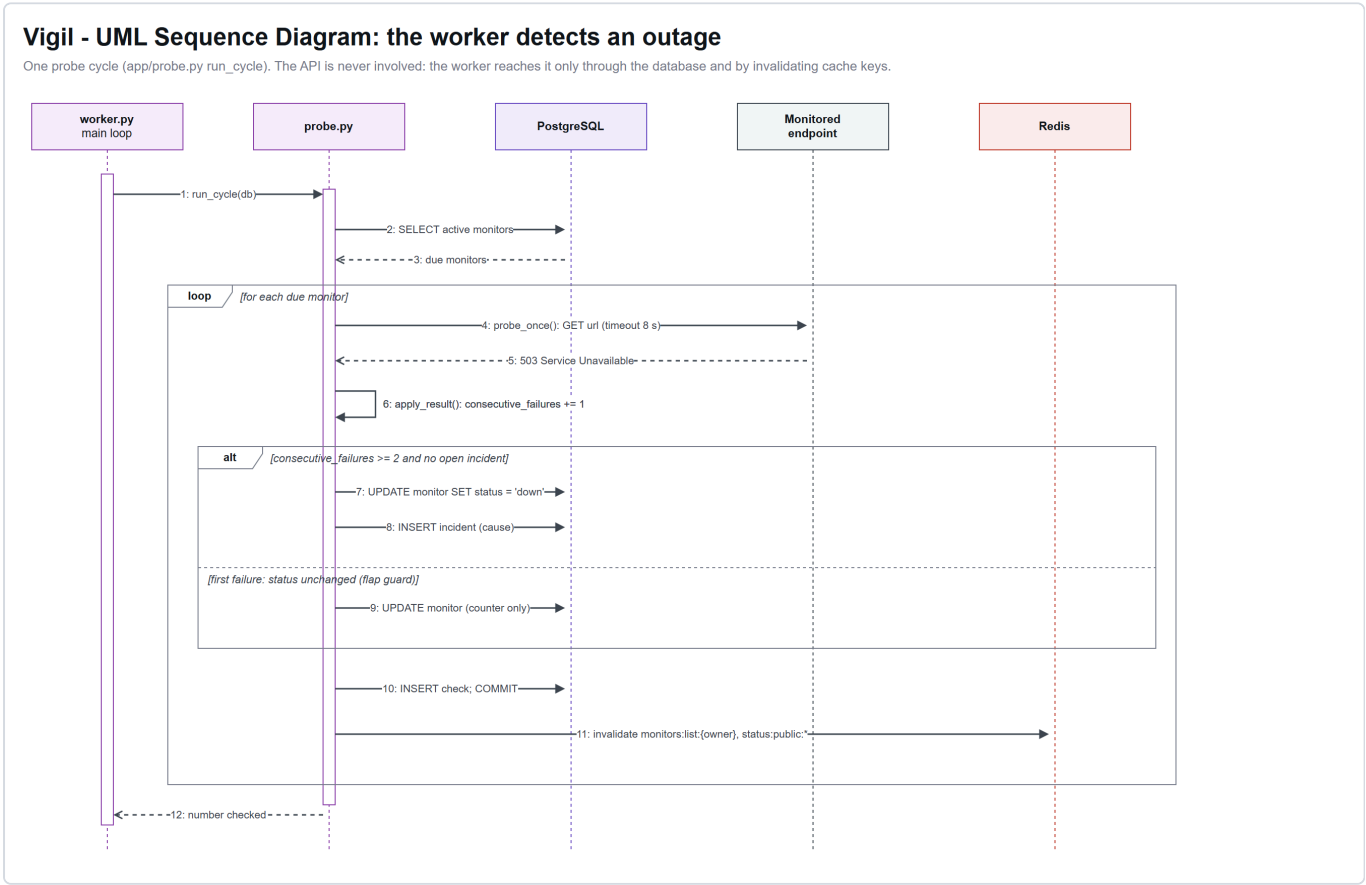


Figure D.3 — Sequence diagram: the worker detects an outage. Editable source: docs/design/diagrams/09-sequence-outage.drawio

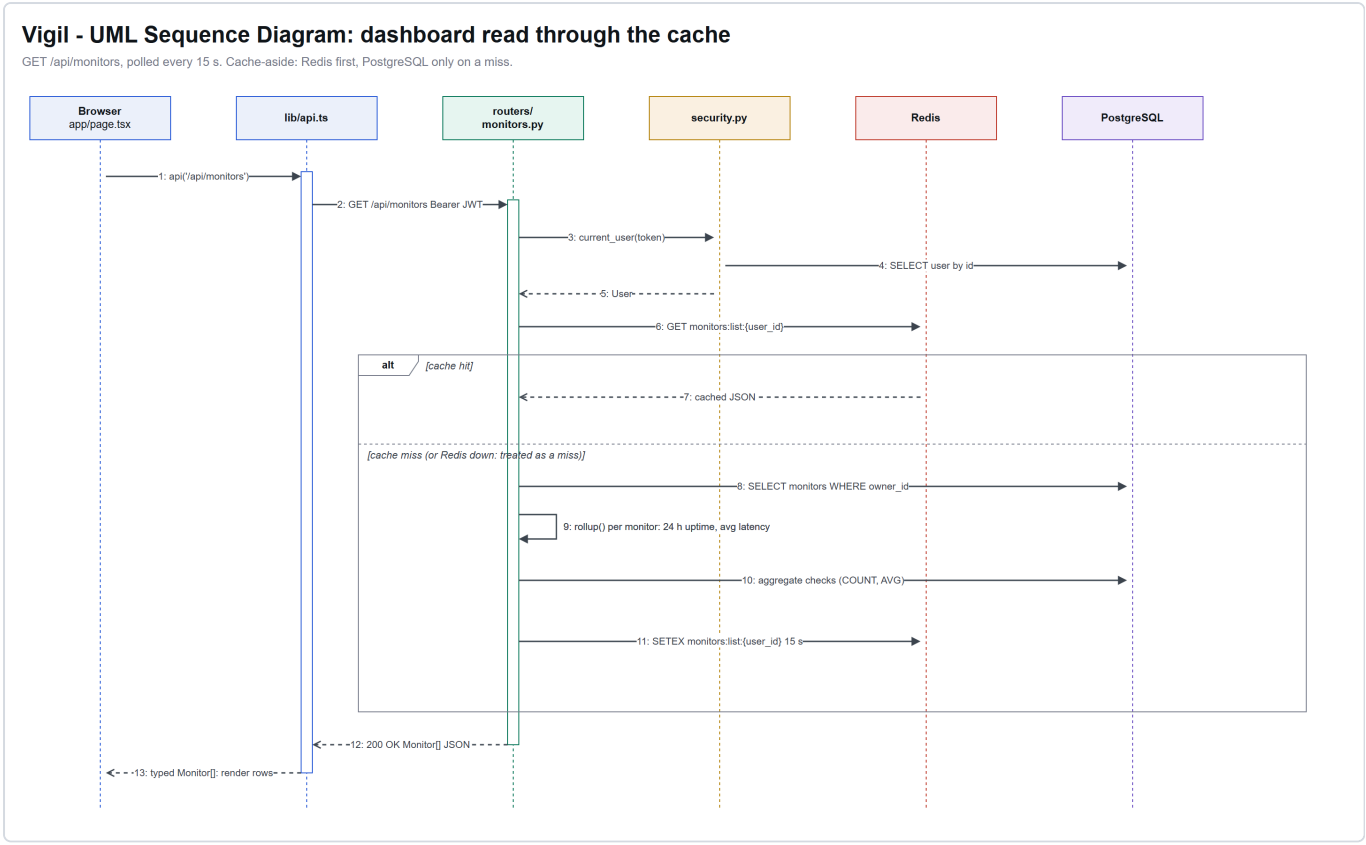


Figure D.4 — Sequence diagram: dashboard read through the cache. Editable source: docs/design/diagrams/10-sequence-dashboard.drawio

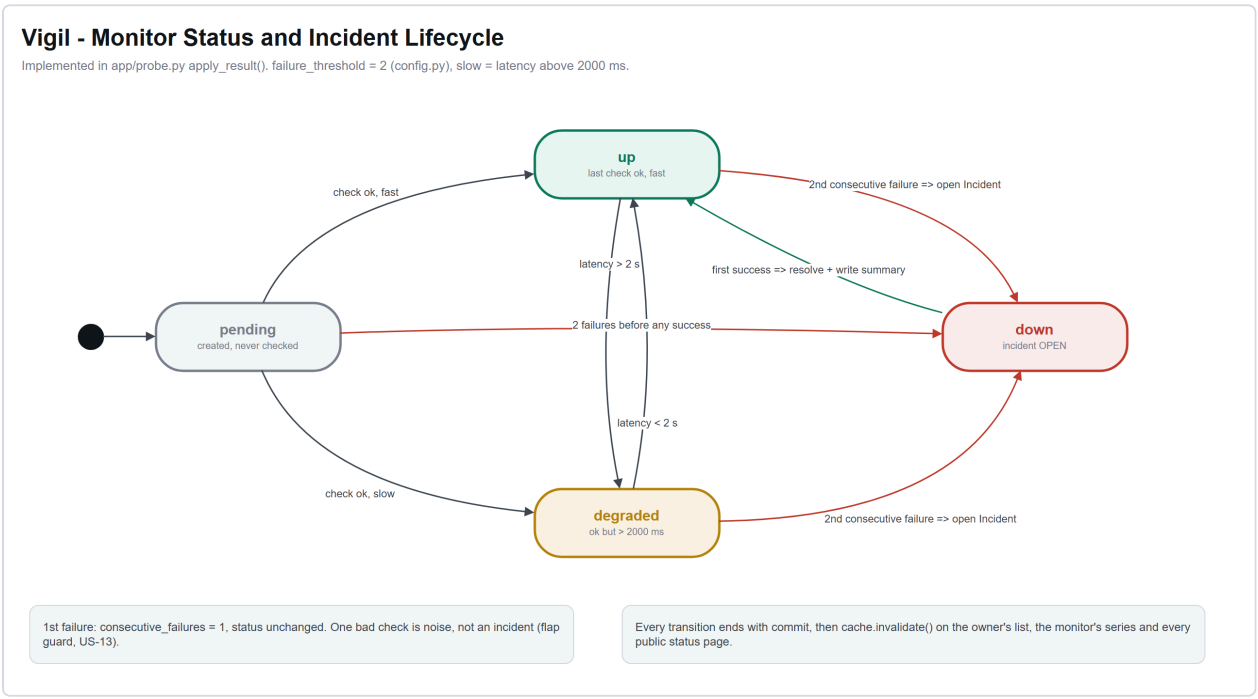


Figure D.5 — State machine diagram: monitor status and incident lifecycle. Editable source: docs/design/diagrams/04-monitor-state-machine.drawio

The component view is Figure 8.2 and the deployment view is Figure 8.1.

Appendix E — Code Listing and GitHub Link

GitHub repository: https://github.com/vatsal-agra/vigil
Release: v0.3.0-rc.1 · **Review 3 pull request:** #28 · **Issues:** 25 user stories

```
vigil/
├── backend/            FastAPI API and probe worker (Python 3.12)
│   ├── app/            main, config, db, models, schemas, security, cache, services, probe, seed
│   │   └── routers/    auth, monitors, incidents, status
│   ├── tests/          unit, integration and regression suites + conftest
│   ├── worker.py       probe loop entrypoint
│   └── Dockerfile      targets: dev · prod · worker
├── frontend/           Next.js 15 + TypeScript (app/ pages, lib/api.ts)
├── e2e/                Playwright end-to-end tests
├── docs/               user stories, wireframes, design/, report/
├── infra/              architecture.drawio, Caddyfile
├── scripts/            GitHub bootstrap, backlog, diagram generators
├── .github/workflows/  ci.yml, mutation.yml
└── docker-compose.yml
```

Area	Files	Lines
Backend application (app/ + worker.py)	17	939
Backend tests	8	905
Frontend (pages, API client, styles)	7	1,014
End-to-end tests	2	103
Scripts (bootstrap, generators)	9	1,797
CI workflows	2	136

E.1 backend/app/probe.py — the checking engine

```python
"""One probe cycle: check due monitors, record results, open/close incidents."""
import logging
import time
from datetime import datetime, timedelta, timezone

import httpx
from sqlalchemy import select
from sqlalchemy.orm import Session

from app import cache
from app.config import settings
from app.models import Check, Incident, Monitor
from app.services import open_incident

log = logging.getLogger("vigil.probe")


def due_monitors(db: Session) -> list[Monitor]:
    now = datetime.now(timezone.utc)
    monitors = db.scalars(select(Monitor).where(Monitor.is_active.is_(True))).all()
    due = []
    for monitor in monitors:
        if monitor.last_checked_at is None:
            due.append(monitor)
            continue
        last = monitor.last_checked_at
        if last.tzinfo is None:
            last = last.replace(tzinfo=timezone.utc)
        if now - last >= timedelta(seconds=monitor.interval_seconds):
            due.append(monitor)
    return due
```

```python
def probe_once(client: httpx.Client, monitor: Monitor) -> Check:
    started = time.perf_counter()
    try:
        response = client.request(
            monitor.method, monitor.url, timeout=settings.probe_timeout_seconds
        )
        latency = (time.perf_counter() - started) * 1000
        return Check(
            monitor_id=monitor.id,
            status_code=response.status_code,
            latency_ms=round(latency, 1),
            ok=response.status_code == monitor.expected_status,
            error=None
            if response.status_code == monitor.expected_status
            else f"Expected {monitor.expected_status}, got {response.status_code}",
        )
    except httpx.TimeoutException:
        return Check(
            monitor_id=monitor.id,
            status_code=None,
            latency_ms=round((time.perf_counter() - started) * 1000, 1),
            ok=False,
            error=f"No response within {settings.probe_timeout_seconds:.0f}s",
        )
    except httpx.HTTPError as exc:
        return Check(
            monitor_id=monitor.id,
            status_code=None,
            latency_ms=None,
            ok=False,
            error=f"Connection failed: {type(exc).__name__}",
        )


def apply_result(db: Session, monitor: Monitor, check: Check) -> None:
    monitor.last_checked_at = datetime.now(timezone.utc)

    if check.ok:
        monitor.consecutive_failures = 0
        slow = check.latency_ms is not None and check.latency_ms > 2000
        monitor.current_status = "degraded" if slow else "up"
        incident = open_incident(db, monitor.id)
        if incident is not None:
            incident.resolved_at = datetime.now(timezone.utc)
            minutes = (incident.resolved_at - incident.started_at).total_seconds() / 60
            incident.summary = (
                f"{monitor.name} recovered after {minutes:.0f} min. "
                f"Trigger: {incident.cause}. Now answering in {check.latency_ms:.0f} ms."
            )
    else:
        monitor.consecutive_failures += 1
        if monitor.consecutive_failures >= settings.failure_threshold:
            monitor.current_status = "down"
            if open_incident(db, monitor.id) is None:
                db.add(
                    Incident(
                        monitor_id=monitor.id,
                        cause=check.error or "check failed",
                        summary=None,
                    )
                )

    db.add(check)
    db.commit()
    cache.invalidate(f"monitors:list:{monitor.owner_id}")
    cache.invalidate(f"monitors:checks:{monitor.id}:*")
    cache.invalidate("status:public:*")


def run_cycle(db: Session, client: httpx.Client | None = None) -> int:
```

```
    """Probe every due monitor once. Pass a client to control the network (tests)."""
    monitors = due_monitors(db)
    if not monitors:
        return 0
    owned = client is None
    client = client or httpx.Client(follow_redirects=True)
    try:
        for monitor in monitors:
            result = probe_once(client, monitor)
            apply_result(db, monitor, result)
            log.info(
                "checked %s -> %s (%s ms)",
                monitor.name,
                "ok" if result.ok else "fail",
                result.latency_ms,
            )
    finally:
        if owned:
            client.close()
    return len(monitors)
```

backend/app/probe.py, lines 1–123

E.2 backend/app/services.py — shared queries

```
"""Shared query helpers used by both the API and the probe worker."""
import re
from datetime import datetime, timedelta, timezone

from sqlalchemy import func, select
from sqlalchemy.orm import Session

from app.models import Check, Incident, Monitor, User


def slugify(value: str) -> str:
    slug = re.sub(r"[^a-z0-9]+", "-", value.lower()).strip("-")
    return slug or "monitor"


def unique_slug(db: Session, owner_id: int, name: str) -> str:
    base = slugify(name)
    candidate, suffix = base, 2
    while db.scalar(
        select(Monitor.id).where(Monitor.owner_id == owner_id, Monitor.slug == candidate)
    ):
        candidate = f"{base}-{suffix}"
        suffix += 1
    return candidate


def rollup(db: Session, monitor_id: int, hours: int = 24) -> dict:
    since = datetime.now(timezone.utc) - timedelta(hours=hours)
    total, passed, avg_latency = db.execute(
        select(
            func.count(Check.id),
            func.count(Check.id).filter(Check.ok.is_(True)),
            func.avg(Check.latency_ms).filter(Check.ok.is_(True)),
        ).where(Check.monitor_id == monitor_id, Check.checked_at >= since)
    ).one()
    return {
        "uptime_24h": round(passed / total * 100, 2) if total else None,
        "avg_latency_ms": round(float(avg_latency), 1) if avg_latency else None,
        "checks_counted": total,
    }


def open_incident(db: Session, monitor_id: int) -> Incident | None:
    return db.scalar(
        select(Incident).where(
            Incident.monitor_id == monitor_id, Incident.resolved_at.is_(None)
        )
```

```
    )


def unique_handle(db: Session, email: str) -> str:
    base = slugify(email.split("@")[0])
    candidate, suffix = base, 2
    while db.scalar(select(User.id).where(User.handle == candidate)):
        candidate = f"{base}-{suffix}"
        suffix += 1
    return candidate
```

backend/app/services.py, lines 1–57

E.3 backend/app/routers/status.py — the cached public status page

```
"""Unauthenticated public status page feed, heavily cached."""
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy import select
from sqlalchemy.orm import Session

from app import cache
from app.db import get_db
from app.models import Monitor, User
from app.services import rollup

router = APIRouter(prefix="/api/status", tags=["status"])

PUBLIC_TTL = 20


@router.get("/{handle}")
def public_status(handle: str, db: Session = Depends(get_db)):
    key = f"status:public:{handle}"
    cached = cache.get_json(key)
    if cached is not None:
        return cached

    user = db.scalar(select(User).where(User.handle == handle))
    if user is None:
        raise HTTPException(status_code=404, detail="No status page at that address.")

    monitors = db.scalars(
        select(Monitor)
        .where(Monitor.owner_id == user.id, Monitor.is_public.is_(True))
        .order_by(Monitor.name)
    ).all()

    services = []
    for monitor in monitors:
        stats = rollup(db, monitor.id)
        services.append(
            {
                "name": monitor.name,
                "slug": monitor.slug,
                "status": monitor.current_status,
                "uptime_24h": stats["uptime_24h"],
                "avg_latency_ms": stats["avg_latency_ms"],
            }
        )

    overall = "operational"
    if any(s["status"] == "down" for s in services):
        overall = "major outage"
    elif any(s["status"] == "degraded" for s in services):
        overall = "degraded"

    payload = {"handle": handle, "overall": overall, "services": services}
    cache.set_json(key, payload, PUBLIC_TTL)
    return payload
```

backend/app/routers/status.py, lines 1–54

E.4 frontend/lib/api.ts — the single HTTP client

```
export async function api<T>(path: string, init: RequestInit = {}): Promise<T> {
  const headers: Record<string, string> = {
    "Content-Type": "application/json",
    ...((init.headers as Record<string, string>) ?? {}),
  };
  const t = token();
  if (t) headers.Authorization = `Bearer ${t}`;

  const res = await fetch(`${API}${path}`, { ...init, headers, cache: "no-store" });
  if (res.status === 401) {
    clearSession();
    if (typeof window !== "undefined") window.location.href = "/login";
    throw new Error("Session expired.");
  }
  if (!res.ok) {
    const body = await res.json().catch(() => ({}));
    throw new Error(body.detail ?? `Request failed (${res.status}).`);
  }
  return res.status === 204 ? (undefined as T) : ((await res.json()) as T);
}
```

frontend/lib/api.ts, lines 53–72

E.5 backend/tests/test_regression.py — the regression suite

```
"""Regression suite: one test per defect fixed or contract that must never break again.

Run on its own with `pytest -m regression`. Each test names the defect or
acceptance criterion it guards, so a red test says what came back.
"""
from datetime import datetime, timedelta

import pytest

from app.models import Incident
from app.probe import apply_result, due_monitors, run_cycle
from helpers import ago, failed

pytestmark = pytest.mark.regression


def test_R1_open_incident_beyond_the_feed_cap_is_still_reported(client, signup, db):
    """Review 2 defect (commit 82be0dd): the dashboard showed 'Open incidents: 0'
    while a monitor was down, because the open incident was older than the newest
    100 rows of the capped feed. The fix relies on ?open_only=true reaching it."""
    auth, _ = signup()
    mid = client.post("/api/monitors", headers=auth, json={"name": "Legacy", "url": "https://l.test/"}).json()
["id"]
    db.add(Incident(monitor_id=mid, cause="still broken", started_at=ago(days=30)))
    db.add_all([Incident(monitor_id=mid, cause=f"blip {n}", started_at=ago(hours=n), resolved_at=ago(hours=n) +
timedelta(minutes=2))
                for n in range(1, 106)])
    db.commit()

    feed = client.get("/api/incidents", headers=auth).json()
    assert len(feed) == 100 and all(i["resolved_at"] for i in feed)      # the cap hides it ...
    [still_open] = client.get("/api/incidents?open_only=true", headers=auth).json()
    assert still_open["cause"] == "still broken"                         # ... the open-only query does not


def test_R2_a_single_failed_check_never_opens_an_incident(db, make_monitor):
    """US-13 AC3, the flap guard: one bad check is noise."""
    m = make_monitor()
    apply_result(db, m, failed(m.id))
    assert db.query(Incident).count() == 0


def test_R3_foreign_resources_are_404_not_403(client, signup):
    """US-04 AC2: a 403 would confirm that the monitor exists."""
```

```
    alice, _ = signup("alice@example.com")
    bob, _ = signup("bob@example.com")
    mid = client.post("/api/monitors", headers=alice, json={"name": "A", "url": "https://a.test/"}).json()["id"]
    assert client.get(f"/api/monitors/{mid}", headers=bob).status_code == 404


def test_R4_naive_timestamps_do_not_crash_the_scheduler(db, make_monitor):
    """due_monitors() must accept a last_checked_at without tzinfo (older rows, other drivers)."""
    m = make_monitor(interval_seconds=60)
    m.last_checked_at = datetime.utcnow() - timedelta(minutes=5)          # naive on purpose
    assert [x.id for x in due_monitors(db)] == [m.id]


def test_R5_the_public_page_never_leaks_urls_or_raw_errors(client, signup, db):
    """US-16 AC3."""
    auth, account = signup()
    client.post("/api/monitors", headers=auth,
                json={"name": "Payments", "url": "https://internal.secret/admin", "is_public": True})
    body = client.get(f"/api/status/{account['handle']}").text
    assert "internal.secret" not in body
    assert "error" not in body and "url" not in body


def test_R6_interval_bounds_are_enforced_on_update_as_well_as_create(client, signup):
    """US-07 AC2: the clamp must hold on PATCH, not only on POST."""
    auth, _ = signup()
    mid = client.post("/api/monitors", headers=auth, json={"name": "A", "url": "https://a.test/"}).json()["id"]
    assert client.patch(f"/api/monitors/{mid}", headers=auth, json={"interval_seconds": 5}).status_code == 422


def test_R7_run_cycle_never_closes_a_client_it_was_given(db, make_monitor, fake_web):
    """Review 3 refactor: run_cycle() accepts an injected client and must leave it open."""
    make_monitor(url="https://a.test/")
    web = fake_web({"https://a.test/": 200})
    run_cycle(db, web)
    assert web.is_closed is False
```

backend/tests/test_regression.py, lines 1–78